RASPBERRY PI 5 · AI HAT 2+ · UNIFIED EDGE INTELLIGENCE · v3.2

# DELTA

## DELTA Plant - AI & Robotics Orchestrator

### per la Salute delle Piante

*Detection and Evaluation of Leaf Troubles and Anomalies*

Visione AI | Pipeline X | Manuale 3.2

## MANUALE UTENTE · v3.2

Copyright © 2026 Paolo Ciccolella. All rights reserved.

Aggiornare con:  python Manuale/genera_manuale.py

Copyright © 2026 Paolo Ciccolella. All rights reserved.

# INDICE

1. Introduzione ..... 3

Scientific Paper ..... 4

2. Hardware — Componenti e configurazione ..... 9

3. Hardware — Modello AI e visione ..... 10

4. Software — Installazione ..... 11

5. Software — Utilizzo del sistema ..... 12

6. Software — API REST e Telegram ..... 13

7. Database e persistenza ..... 14

8. Architettura software — Moduli ..... 15

9. Risoluzione problemi ..... 16

10. Aggiornamento del manuale ..... 17

11. DELTA Academy — Formazione operatore ..... 18

12. Analisi multi-organo — Foglia/Fiore/Frutto ..... 20

13. Oracolo Quantistico di Grover ..... 23

14b. Chat AI — HuggingFace LLM e «Chiedi a DELTA» ..... 26

15. Sicurezza e Pannello Amministratore ..... 28

16. Input immagini da cartella — No-Camera ..... 31

17. Installazione automatica Raspberry Pi 5 ..... 33

18. MLOps — Addestramento e Miglioramento Continuo ..... 36

19. Modello Pre-addestrato — Download automatico ..... 39

20. GitHub Publisher — Pubblicazione automatica ..... 42

Appendice Hardware — Assemblaggio e Schema elettrico ..... 45

Appendice Licenza — DELTA PLANT SOFTWARE LICENSE ..... 48

Copyright © 2026 Paolo Ciccolella. All rights reserved.

# 1. INTRODUZIONE

DELTA (Detection and Evaluation of Leaf Troubles and Anomalies) è un sistema di intelligenza artificiale per il monitoraggio continuo della salute delle piante. Combina sensori ambientali, visione artificiale e modelli di deep learning per fornire diagnosi fitosanitarie in tempo reale direttamente su Raspberry Pi 5.

Copyright © 2026 Paolo Ciccolella. All rights reserved.

# Appendice Licenza — DELTA PLANT SOFTWARE LICENSE

Copyright © 2026 Paolo Ciccolella. All rights reserved.

Software Release: v3.2

(This release field must be updated at each official software release.)

Il testo integrale della licenza e disponibile nel file LICENSE nella directory radice del repository GitHub. GitHub visualizza automaticamente la licenza nel pannello informativo del progetto.

```
> POSIZIONE FILE LICENSE
DELTA-PLANT/
  LICENSE   <- testo integrale della licenza
```

## 1. Core System (Proprietary)

The DELTA PLANT core system, including but not limited to:

- main control software
- sensor management system
- automation engine
- safety and execution modules

is proprietary software.

You are NOT permitted to:

- copy, modify, distribute, or reverse engineer the core system
- use the core system outside the authorized DELTA ecosystem
- remove or alter copyright notices

Any unauthorized use of the core system is strictly prohibited.

## 2. Modules / Extensions (Open Source Components)

Certain modules, plugins, or SDK components of DELTA PLANT may be released under open-source licenses (such as MIT or Apache 2.0).

These components are clearly marked in their respective directories.

For open-source modules:

- you may use, modify, and distribute them
- you must respect the license specified in each module
- attribution to the original author is required

## 3. AI Services and Cloud Features

Any AI-related services, cloud APIs, or external model integrations (including but not limited to language model processing) are provided as a service layer and are NOT part of the open-source components.

These services may:

- require authentication

- be subject to usage limits
- be modified or discontinued at any time

## 4. Liability Disclaimer

**[!] ATTENZIONE**

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND.

The author is not responsible for:

- hardware damage
- data loss
- incorrect automation behavior
- misuse of the system

## 5. Scientific Research Use

Scientific and academic research use of the DELTA PLANT core system is allowed only for non-commercial purposes, subject to all terms in this license.

For research use, you must:

- keep all copyright and license notices intact
- provide clear attribution to the author in publications and reports
- use the software in compliance with applicable laws and ethical standards
- avoid redistributing proprietary core code or derivative proprietary builds

Research use does NOT grant ownership or relicensing rights on the DELTA PLANT proprietary core system.

## 6. Commercial Use

Commercial use of the DELTA PLANT core system requires a separate written license agreement with the author.

Open-source modules may be used commercially according to their respective licenses.

## 7. Final Terms

By using any part of DELTA PLANT, you agree to these terms.

Violation of this license may result in termination of usage rights and legal action.

---

La release v3.2 consolida DELTA come piattaforma edge AI ibrida: backend MobileNetV2 stabile in produzione, backend EfficientFormerV2-S1 int8 realmente eseguibile su questo hardware, Pipeline X resumable end-to-end, telemetria locale, memoria diagnostica persistente e manualistica rigenerabile automaticamente a ogni rilascio.

## Caratteristiche principali — v3.2

- Diagnostica fogliare PlantVillage a 33 classi con backend MobileNetV2 + EfficientFormerV2-S1
- EfficientFormerV2-S1 int8 validato on-device, con fallback automatico float32 e explainability LayerCAM
- Pipeline X resumable: fine-tuning, export, evaluation, benchmark, dissemination e rigenerazione del manuale PDF
- Memoria persistente per chat e diagnosi, utile per follow-up coerenti anche dopo riavvio
- Lettura continua di 7 parametri ambientali via I2C o inserimento manuale guidato

Copyright © 2026 Paolo Ciccolella. All rights reserved.

21 regole esperte agronomiche + Oracolo Quantistico di Grover per il Quantum Risk Score [0,1]

- DELTA Academy e Lab MLOps per formazione operatore e miglioramento continuo
- Pannello amministratore, API REST opzionale, export Excel e materiali divulgativi automatici
- Installazione automatica su Raspberry Pi 5 con autostart systemd e preflight AI

## Come usare questo manuale

Il manuale è diviso in due parti principali: la Parte A descrive l'hardware (componenti, collegamento, configurazione fisica); la Parte B descrive il software (installazione, avvio, utilizzo delle funzioni, pipeline MLOps, interpretazione degli output e aggiornamento della documentazione di release).

Copyright © 2026 Paolo Ciccolella. All rights reserved.

SCIENTIFIC PAPER

# DELTA: A Real-Time Multi-Organ Plant Health Diagnosis System Integrating Computer Vision, Environmental Sensing, and Grover Quantum Oracle Risk Quantification on Raspberry Pi 5 with AI HAT 2+

*DELTA Project — Agricultural AI Research*

*Raspberry Pi Foundation Compatible Platform*

## Abstract

We present DELTA (Detection and Evaluation of Leaf Troubles and Anomalies), an embedded AI system designed for real-time plant health monitoring in precision agriculture. DELTA runs on Raspberry Pi 5 paired with the AI HAT 2+ neural processing unit and integrates three complementary diagnostic layers: (i) multi-organ computer vision (leaf, flower, and fruit segmentation via adaptive HSV colour-space analysis), (ii) multi-sensor environmental monitoring (temperature, humidity, atmospheric pressure, illuminance, CO2, pH, and electrical conductivity), and (iii) a classical simulation of Grover's quantum search algorithm used as a composite Quantum Risk Oracle (QRO). The QRO encodes 16 agronomic risk states in a 4-qubit register, applies a phase-flip oracle that marks active adverse states, and performs Grover diffusion to amplify their probability amplitudes, yielding a normalised Quantum Risk Score (QRS in [0, 1]). The system detects 21 distinct pathological conditions across leaves, flowers and fruits, generates structured agronomic recommendations, and persists all records to a local SQLite database with Excel export capability. Experimental results on simulated multi-factor scenarios demonstrate that the QRO correctly identifies the dominant risk category with quadratic amplitude amplification and provides an interpretable compound risk metric superior to scalar rule-based aggregation. DELTA is fully operational offline on low-cost embedded hardware, making it suitable for deployment in resource-constrained agricultural environments.

*Keywords: precision agriculture, computer vision, plant disease detection, quantum computing simulation, Grover's algorithm, edge AI, Raspberry Pi, multi-organ analysis, risk quantification, embedded systems.*

## 1. Introduction

Plant diseases account for an estimated 20-40% of global crop losses annually, with cascading effects on food security and agricultural economies [1]. Early, accurate detection of pathological conditions — encompassing leaves, flowers, and fruits — is therefore critical for timely agronomic intervention. While convolutional neural network (CNN)-based leaf disease classification has achieved state-of-the-art accuracy on benchmarks such as PlantVillage [2], deployed solutions rarely integrate multi-organ analysis, real-time environmental sensing, or principled composite risk aggregation. Furthermore, edge deployment on low-cost hardware remains an open challenge due to computational constraints.

Quantum computing offers algorithmic tools with potential speedups over classical approaches for search and optimisation problems. Grover's algorithm [3] achieves O(sqrt(N)) query complexity for unstructured search, compared to the classical O(N), providing a quadratic speedup. While current quantum hardware remains noisy and limited in scale, classical simulations of quantum algorithms can already be exploited for their computational semantics — in particular, amplitude amplification — as a powerful metaphor and mechanism for priority-aware risk ranking in multi-factor diagnostic systems.

In this paper, we introduce DELTA, a modular platform that combines edge AI inference, multi-organ computer vision,

Copyright © 2026 Paolo Ciccolella. All rights reserved.

environmental sensor fusion, and a classically-simulated Grover Quantum Oracle for composite risk quantification. The system is implemented in Python 3.11+ and runs in real time on Raspberry Pi 5 with a dedicated neural processing unit (NPU).

## 2. System Architecture

DELTA follows a modular pipeline architecture with six interconnected layers:

| | |
|---|---|
| **Layer 1 — Sensing** | Seven environmental parameters are acquired at configurable intervals via I2C sensors: temperature (BME680), relative humidity (BME680), atmospheric pressure (BME680), illuminance (VEML7700), CO2 concentration (SCD41), substrate pH and electrical conductivity (ADS1115 ADC). Raw readings are smoothed with a sliding-window moving average (n=5) and validated against agronomic thresholds. |
| **Layer 2 — Imaging** | A Raspberry Pi Camera Module (1920x1080 px, 30 fps) acquires plant frames. Pre-processing includes Gaussian blur, histogram equalisation, and bilinear resizing to 224x224 px for NPU inference. |
| **Layer 3 — Multi-Organ Segmentation** | PlantOrganDetector performs HSV colour-space segmentation across three independent organ classes: leaf (green, single range), flower (5 chromatic ranges covering yellow, white, pink, red), and fruit (5 ranges for red, orange, yellow-mature, and green-fruit). Each organ is characterised by a coverage ratio and bounding-box set; organ presence is confirmed if coverage exceeds a configurable detection threshold (default 15%). |
| **Layer 4 — AI Inference** | The primary model is a TFLite float16 network with float32 input (MobileNetV2 preprocessing). Inference runs on Raspberry Pi 5 via XNNPACK and can leverage the AI HAT stack when available. The default diagnostic set is leaf-only (33 classes). Flower/fruit classifiers remain optional and can be re-enabled via configuration. Low-confidence predictions (< 50%) trigger active-learning human review requests. |
| **Layer 5 — Rule-Based Diagnosis** | A 21-rule expert system evaluates sensor thresholds and AI outputs in combination. Rules are partitioned into: environmental (6 rules), AI-driven (2 rules), floral pathology (4 rules), and fruit pathology (5 rules) plus legacy leaf rules. Activated rules are ranked by a 5-level risk priority schema (none/low/medium/high/critical). |
| **Layer 6 — Quantum Risk Oracle** | The GroverRiskOracle encodes activated rule states as adverse quantum states and applies Grover amplitude amplification to produce a Quantum Risk Score (QRS). Full details in Section 3. |

## 3. The Grover Quantum Risk Oracle

Let H denote the 4-qubit Hilbert space with basis states |0>, |1>, ..., |15>, each representing one agronomic risk category (Table 1). The oracle operates as follows:

| | |
|---|---|
| **Step 1 — Initialisation** | Prepare the uniform superposition via the Hadamard transform: \|psi_0> = H^(x4)\|0>^(x4) = (1/sqrt(N)) * sum_i \|i>, N=16. All risk states are equally |

Copyright © 2026 Paolo Ciccolella. All rights reserved.

| | |
|---|---|
| | weighted, representing maximum a priori uncertainty. |
| **Step 2 — Oracle U_f** | Mark adverse states S subset {0,...,15} with a phase flip: U_f\|i> = -\|i> if i in S, else \|i>. S is determined by mapping activated diagnostic rule IDs to quantum state indices. |
| **Step 3 — Grover Diffusion U_s** | Apply the inversion-about-the-mean operator: U_s = 2\|psi><psi\| - I. Implemented as: new_state[i] = 2*mean(state) - state[i]. This amplifies the amplitude of marked states and suppresses unmarked ones. |
| **Step 4 — Iteration** | Repeat (U_s * U_f) for k iterations, where k = min(k_cfg, round(pi/4 * sqrt(N/\|S\|))). The optimal k maximises the probability of measuring a marked state. |
| **Step 5 — Measurement** | Compute probability distribution P_i = \|alpha_i\|^2, normalised. The Quantum Risk Score QRS = sum_{i in S} P_i * w_i, where w_i are category-specific risk weights in [0.4, 0.95]. A compound synergy bonus is applied when \|S\| >= 3, reflecting the non-linear interaction of multiple simultaneous stressors. |

The QRS is mapped to five risk levels: none (QRS < 0.25), low (0.25-0.45), medium (0.45-0.65), high (0.65-0.80), and critical (QRS >= 0.80). The dominant adverse state (argmax P) provides the primary diagnostic focus. The amplification gain G = P_dominant / (1/N) quantifies the benefit of Grover search over uniform random selection.

## 4. Experimental Evaluation

We evaluated the QRO on a synthetic multi-factor stress scenario representative of severe agronomic conditions: concurrent fungal risk (FUNG_01: RH=82%, T=29C), Botrytis on flower (FLOW_04), AI-detected leaf disease (AI_01, conf.=0.87), temperature stress (TEMP_01), and floral disease (FLOW_03). Results are summarised below:

| | |
|---|---|
| **Active adverse states \|S\|** | 5 (states 0, 10, 12, 13, 13*) |
| **Grover iterations k** | 1 (optimal for \|S\|/N = 5/16) |
| **Dominant state** | \|0000> Fungal risk (FUNG_01) — P=0.312 |
| **Quantum Risk Score (QRS)** | 0.885 — CRITICAL level |
| **Amplification gain G** | 5.0x over uniform baseline |
| **Rule-based aggregate risk** | CRITICAL (max-priority rule: FLOW_04) |
| **Compound synergy bonus applied** | Yes (\|S\|=5 >= 3 threshold) |
| **Recommendations generated** | 7 across categories: fungal, flower, irrigation, pathology, quantum_risk, soil, co2 |
| **Inference latency (NPU)** | < 50 ms per organ (target hardware) |
| **Total diagnosis cycle time** | ~2-4 s including sensor read + 3-organ inference |

The QRO correctly identified the fungal risk state as dominant after amplitude amplification, consistent with the highest risk weight (w=0.8) and highest co-occurrence frequency in our rule set. The compound synergy bonus raised the QRS from a naive weighted sum of 0.72 to 0.885, reflecting the known synergistic effect of concurrent high humidity and floral susceptibility.

## 5. Discussion

DELTA demonstrates that a classical simulation of Grover's algorithm, while not providing the quantum speedup of actual quantum hardware, provides a principled and interpretable framework for composite risk quantification in multi-factor agricultural diagnostics. Key advantages over conventional rule aggregation include: (a) amplitude

Copyright © 2026 Paolo Ciccolella. All rights reserved.

amplification produces a smooth, continuous risk measure rather than a discrete maximum; (b) the superposition metaphor naturally captures diagnostic uncertainty at the initialisation stage; (c) the compound synergy term is a natural consequence of the diffusion operator when multiple states are marked, not an ad-hoc correction; (d) the framework is extensible — adding new risk states requires only extending the state-rule mapping without modifying the core algorithm.

Limitations include the fixed 4-qubit register (16 states), which may be insufficient for crops with highly diverse pathology profiles. Future work will explore dynamic qubit scaling, integration with real quantum backends (IBM Quantum, IonQ) via Qiskit, and training organ-specific TFLite models on domain-specific agricultural datasets.

## 6. Conclusions

We presented DELTA, an embedded, real-time plant health diagnosis system that uniquely combines multi-organ computer vision, seven-parameter environmental sensing, a 21-rule expert system, and a classically-simulated Grover Quantum Oracle for composite risk scoring. The system operates entirely offline on Raspberry Pi 5 + AI HAT 2+, achieving diagnosis cycles of 2-4 seconds with seven environmental parameters and three-organ visual analysis. DELTA is designed for extensibility, providing a practical platform for research at the intersection of precision agriculture, edge AI, and quantum-inspired computing.

## References


[1] FAO (2021). The State of Food and Agriculture 2021. Food and Agriculture Organization of the United Nations, Rome.

[2] Hughes, D., Salath, M. (2016). An open access repository of images on plant health to enable the development of mobile disease diagnostics. arXiv:1511.08060.

[3] Grover, L.K. (1996). A fast quantum mechanical algorithm for database search. Proceedings of the 28th ACM Symposium on Theory of Computing, pp. 212-219. ACM, New York.

[4] Mohanty, S.P., Hughes, D.P., Salath, M. (2016). Using deep learning for image-based plant disease detection. Frontiers in Plant Science, 7, 1419.

[5] Nielsen, M.A., Chuang, I.L. (2010). Quantum Computation and Quantum Information (10th anniversary ed.). Cambridge University Press.

[6] Bauer, B. et al. (2020). Quantum algorithms for quantum chemistry and quantum materials science. Chemical Reviews, 120(22), 12685-12717.

[7] Ferentinos, K.P. (2018). Deep learning models for plant disease detection and diagnosis. Computers and Electronics in Agriculture, 145, 311-318.

[8] Raspberry Pi Foundation (2024). Raspberry Pi 5 Technical Reference Manual. Available: https://www.raspberrypi.com/documentation/

[9] Hailo Technologies (2024). Hailo-8 AI Processor Datasheet v2.1. Available: https://hailo.ai/products/ai-accelerators/

[10] Abadi, M. et al. (2016). TensorFlow: Large-scale machine learning on heterogeneous systems. arXiv:1603.04467.



Copyright © 2026 Paolo Ciccolella. All rights reserved.

# 2. PARTE A — HARDWARE: COMPONENTI E CONFIGURAZIONE

## 2.1 Lista componenti

| | |
|---|---|
| **Raspberry Pi 5** | 16 GB RAM — scheda principale |
| **AI HAT 2+** | Acceleratore NPU 40 TOPS per inferenza TFLite |
| **Pi Camera Module 3** | Fotocamera 12 MP autofocus — acquisizione foglie |
| **Sensore BME680** | Temperatura, Umidità relativa, Pressione, Gas VOC |
| **Sensore VEML7700** | Luminosità ambientale (lux) — interfaccia I2C |
| **Sensore SCD41** | CO2 (400–5000 ppm) — fotoacustico NDIR |
| **ADC ADS1115** | Convertitore A/D 16-bit a 4 canali — per pH ed EC |
| **Elettrodo pH** | Sonda potenziometrica pH 0–14 |
| **Cella EC** | Sensore conducibilità elettrica 0–5 mS/cm |
| **Alimentatore** | Alimentatore originale Raspberry Pi USB-C 5V/5A — minimo 27W consigliati |
| **Memoria di archiviazione** | MicroSD o SSD esterna da 256 GB — consigliata per OS e dati |

## 2.2 Schema di collegamento I2C

Tutti i sensori comunicano tramite bus I2C sul connettore GPIO a 40 pin di Raspberry Pi. Utilizzare cavi Dupont da 10–20 cm per connessioni stabili.

| | |
|---|---|
| **BME680 (Temp/Umid/Press)** | I2C bus 1 — indirizzo 0x76 |
| **VEML7700 (Luminosità)** | I2C bus 1 — indirizzo 0x10 |
| **SCD41 (CO2)** | I2C bus 1 — indirizzo 0x62 |
| **ADS1115 (ADC pH/EC)** | I2C bus 1 — indirizzo 0x48 |

Connessioni GPIO standard:

- SDA → GPIO 2 (Pin 3)
- SCL → GPIO 3 (Pin 5)
- VCC → Pin 1 (3.3V) o Pin 4 (5V) in base al sensore
- GND → Pin 6, 9, 14, 20, 25, 30, 34 o 39

## 2.3 AI HAT 2+ — installazione

- Allineare il connettore M.2 dell'AI HAT con lo slot sulla Pi 5
- Inserire delicatamente e fissare con la vite M2.5 in dotazione
- Collegare il cavo FFC dal connettore CSI/AI HAT per alimentazione NPU
- Verificare il riconoscimento: sudo lspci | grep -i hailo
- Installare driver Hailo RT seguendo: https://hailo.ai/developer-zone/

## 2.4 Pi Camera Module 3

- Aprire il blocco del connettore CSI sul Raspberry Pi 5 (connettore bianco)
- Inserire il cavo flat con i contatti metallici verso il basso
- Bloccare il connettore premendo verso il basso
- Verificare: libcamera-hello --list-cameras

Copyright © 2026 Paolo Ciccolella. All rights reserved.

## 2.5 Calibrazione sensori elettrochimici

pH — calibrazione a due punti (eseguire prima di ogni sessione di misura):

- Immergere l'elettrodo in soluzione buffer pH 7.0 → annotare la tensione V1
- Risciacquare con acqua distillata
- Immergere in soluzione buffer pH 4.0 → annotare la tensione V2
- Aggiornare la funzione _voltage_to_ph() in sensors/reader.py con i nuovi coefficienti

EC — calibrazione con soluzione standard (es. 1.413 mS/cm a 25°C):

- Immergere la cella nella soluzione standard
- Misurare la tensione con un multimetro
- Aggiornare il fattore lineare in _voltage_to_ec() in sensors/reader.py

## 2.6 Parametri operativi (da config.py)

Soglie agronomiche attualmente configurate nel sistema:

| | |
|---|---|
| **Temperatura min/max** | 5.0 / 40.0 °C |
| **Temperatura ottimale** | 18.0 – 28.0 °C |
| **Umidità min/max** | 20.0 / 95.0 % |
| **Umidità ottimale** | 40.0 – 70.0 % |
| **Soglia rischio fungino** | 80.0 % |
| **Luminosità min fotosintesi** | 1000.0 lux |
| **Luminosità ottimale** | 25000.0 lux |
| **Stress luce alta** | 80000.0 lux |
| **CO2 min/max** | 300.0 / 5000.0 ppm |
| **CO2 ottimale** | 400.0 – 1500.0 ppm |
| **pH ottimale suolo** | 6.0 – 7.0 |
| **EC ottimale (mS/cm)** | 0.8 – 2.5 |
| **EC tossica** | > 3.5 mS/cm |
| **Intervallo lettura** | ogni 30 secondi |
| **Finestra smoothing** | 5 campioni |

Copyright © 2026 Paolo Ciccolella. All rights reserved.

# 3. PARTE A (cont.) — MODELLO AI E VISIONE

## 3.1 Modello di classificazione

DELTA utilizza una stack multi-backend per la classificazione fogliare edge. Il profilo baseline resta MobileNetV2 in TensorFlow Lite float16, mentre il profilo avanzato introduce EfficientFormerV2-S1 come backend ibrido CNN/ViT, attivabile da MODELS_REGISTRY o config.yaml, con ensemble probabilistico e spiegabilita LayerCAM. In v3.2 la variante EfficientFormer int8 e validata sull'hardware target e il runtime puo fare fallback automatico a float32 se la variante richiesta non e allocabile. Entrambi i backend lavorano su input 224x224 con preprocessing MobileNet-like nel range [-1, 1].

| | |
|---|---|
| **Percorso modello** | /home/proctor81/Desktop/DELTA-PLANT/models/plant_disease_model_39classes.tflite |
| **Percorso etichette** | /home/proctor81/Desktop/DELTA-PLANT/models/labels_33classes_correct.txt |
| **Dimensione input** | (224, 224) pixel |
| **Tipo dato input** | float32 |
| **Soglia confidenza** | 65% |
| **Soglia active learning** | 50% |
| **Thread inferenza** | 4 |
| **Edge TPU / AI HAT** | Abilitato |

## 3.1b Backend avanzato EfficientFormerV2-S1

Il backend EfficientFormerV2-S1 e progettato per portare DELTA verso un profilo edge-AI piu spiegabile e piu robusto su classi morfologicamente simili. Il backend supporta una variante int8 fully quantized validata su questo hardware, ensemble con MobileNetV2, explainability LayerCAM e fallback runtime float32 per evitare blocchi operativi durante il deploy.

| | |
|---|---|
| **Backend** | efficientformer |
| **Display name** | EfficientFormerV2-S1 |
| **Variante runtime predefinita** | int8 |
| **Thread target** | 6 |
| **Ensemble** | Sì |
| **Modello ensemble** | generale |
| **Explainability** | layercam |
| **Checkpoint PyTorch** | /home/proctor81/Desktop/DELTA-PLANT/models/efficientformer_v2_s1_33classes.pth |
| **Fallback runtime** | float32 automatico se la variante richiesta non alloca |

## 3.1c Come leggere LayerCAM

LayerCAM e una tecnica di explainability visuale: genera una heatmap che evidenzia le regioni dell'immagine che hanno contribuito maggiormente alla decisione del modello. In DELTA la heatmap viene sovrapposta alla foto originale e puo essere inviata su Telegram o salvata nei file di explainability.

- Colori piu caldi (giallo/rosso) indicano un contributo maggiore alla predizione; colori piu freddi indicano contributo minore.

Copyright © 2026 Paolo Ciccolella. All rights reserved.

L'overlay serve a verificare se il modello sta osservando lesioni, bordi necrotici, aree clorotiche, nervature o altre strutture coerenti con il referto.

- LayerCAM non e una segmentazione clinica del tessuto malato: non delimita con precisione millimetrica la patologia e non sostituisce la valutazione agronomica.
- In v3.2 DELTA puo usare EfficientFormer come motore explainability anche quando la classificazione operativa resta sul backend generale, se la spiegabilita e disponibile.

## 3.2 Classi diagnostiche

1. Apple_Apple_scab
2. Apple_Black_rot
3. Apple_Cedar_apple_rust
4. Apple_healthy
5. Bell_pepper_Bacterial_spot
6. Bell_pepper_healthy
7. Blueberry_healthy
8. Cherry_Powdery_mildew
9. Cherry_healthy
10. Corn_Cercospora
11. Corn_Common_rust
12. Corn_Northern_Leaf_Blight
13. Corn_healthy
14. Grape_Black_rot
15. Grape_Esca
16. Grape_Leaf_blight
17. Grape_healthy
18. Peach_healthy
19. Potato_Early_blight
20. Potato_Late_blight
21. Potato_healthy
22. Squash_Powdery_mildew
23. Strawberry_Leaf_scorch
24. Strawberry_healthy
25. Tomato_Bacterial_spot
26. Tomato_Early_blight
27. Tomato_Late_blight
28. Tomato_Leaf_Mold
29. Tomato_Septoria_leaf_spot
30. Tomato_Target_Spot
31. Tomato_Yellow_Leaf_Curl
32. Tomato_healthy
33. Tomato_mosaic_virus

## 3.3 Parametri camera

| | |
|---|---|
| **Indice videocamera** | 0 |

Copyright © 2026 Paolo Ciccolella. All rights reserved.

| | |
|---|---|
| **Risoluzione cattura** | 1920 × 1080 px |
| **Risoluzione preview** | 640 × 480 px |
| **Formato** | MJPG |
| **FPS** | 30 |
| **Metodo segmentazione** | hsv |
| **Area minima foglia** | 500 pixel |
| **Salvataggio catture** | Sì |
| **Directory catture** | /home/proctor81/Desktop/DELTA-PLANT/datasets/captures |

Copyright © 2026 Paolo Ciccolella. All rights reserved.

# 4. PARTE B — SOFTWARE: INSTALLAZIONE

## 4.1 Prerequisiti di sistema

- Raspberry Pi OS (64-bit) — versione Bookworm o superiore
- Python 3.12 consigliato (compatibilità TensorFlow/TFLite)
- Evitare Python >= 3.13 per runtime AI — le wheel di TensorFlow/TFLite possono mancare
- Git installato: sudo apt install git
- I2C abilitato: sudo raspi-config → Interface Options → I2C → Abilita
- Camera abilitata: sudo raspi-config → Interface Options → Camera → Abilita

## 4.2 Clonazione del progetto

> TERMINALE

```
# Clona il repository
git clone <URL_REPOSITORY> ~/DELTA
cd ~/DELTA

# Crea ambiente virtuale — usare Python 3.10–3.12
python3.12 -m venv .venv   # Raspberry Pi / Linux
/opt/homebrew/bin/python3.12 -m venv .venv  # macOS (Homebrew)
source .venv/bin/activate
```

## 4.3 Dipendenze Python

Le seguenti librerie sono richieste (lette automaticamente da requirements.txt):

- opencv-python-headless>=4.8.0
- numpy>=1.24.0
- pandas>=2.0.0
- PyYAML>=6.0
- openpyxl>=3.1.0
- fpdf2>=2.7.0
- scikit-learn>=1.3.0
- ai-edge-litert==1.2.0
- flask>=3.0.0
- fastapi>=0.115.0
- uvicorn[standard]>=0.30.0
- httpx>=0.27.0
- pydantic>=2.7.0
- pydantic-settings>=2.3.4
- python-jose[cryptography]>=3.3.0
- cryptography>=42.0.0
- slowapi>=0.1.9
- bleach>=6.1.0
- python-dotenv>=1.0.1
- redis>=5.0.0

Copyright © 2026 Paolo Ciccolella. All rights reserved.

reportlab>=4.2.0

- shapely>=2.0.4
- tifffile>=2024.5.22
- zstandard>=0.23.0
- pykrige>=1.7.0
- scikit-image>=0.22.0
- huggingface_hub>=0.27.0
- python-telegram-bot[job-queue,rate-limiter]>=20.7
- requests>=2.31.0
- faster-whisper>=1.0.3
- piper-tts>=1.4.2
- elevenlabs>=1.7.0
- edge-tts>=6.1.12
- pydub>=0.25.1

> TERMINALE

```
# Installa tutte le dipendenze
pip install -r requirements.txt

# Su Raspberry Pi — installa anche le librerie hardware:
pip install adafruit-circuitpython-bme680 \
            adafruit-circuitpython-veml7700 \
            adafruit-circuitpython-scd4x \
            adafruit-circuitpython-ads1x15 \
            picamera2 RPi.GPIO
```

## 4.4 Installazione TFLite Runtime

Su Raspberry Pi 5 (aarch64) il runtime raccomandato è ai-edge-litert==1.2.0. Versioni >= 1.3.0 causano un segmentation fault durante l'import del runtime nativo su BCM2712. Se ai-edge-litert non è disponibile, usare TensorFlow 2.21.0 come fallback.

> TERMINALE

```
# Su Raspberry Pi 5 (aarch64, Python 3.12) — RACCOMANDATO
pip install ai-edge-litert==1.2.0

# NOTA: NON installare versioni >= 1.3.0 su RPi5 — causano segfault (BCM2712)
# pip install ai-edge-litert  <- NON fare questo

# Fallback — se ai-edge-litert non è disponibile:
pip install tensorflow==2.21.0 flatbuffers==25.12.19

# Opzione con supporto Edge TPU / AI HAT 2+ (pycoral)
# Seguire: https://coral.ai/docs/accelerator/get-started/
# pip install pycoral
```

**Compatibilità Python runtime AI — RPi5 aarch64**

Python 3.12 è raccomandato. ai-edge-litert >= 1.3.0 causa segfault su Raspberry Pi 5 (SoC BCM2712, aarch64): usare SEMPRE ai-edge-litert==1.2.0. tflite-runtime non è disponibile su aarch64/Python 3.12 via pip. Con Python >= 3.13 anche le wheel tensorflow

Copyright © 2026 Paolo Ciccolella. All rights reserved.

sono assenti.

## 4.5 Installazione driver AI HAT 2+ (Hailo)

> TERMINALE

```
# Aggiungi il repository Hailo
sudo apt install hailo-all
sudo reboot

# Verifica dopo il riavvio
hailortcli fw-control identify
```

Copyright © 2026 Paolo Ciccolella. All rights reserved.

# 5. PARTE B (cont.) — UTILIZZO DEL SOFTWARE

## 5.1 Avvio del sistema

Il modo più semplice per avviare DELTA è il comando globale 'delta', installato in /usr/local/bin/ durante il setup. Basta aprire un terminale e digitare 'delta' da qualsiasi directory. In alternativa è possibile avviare AVVIO_DELTA.py direttamente dalla cartella del progetto.

> TERMINALE

```
# Metodo più semplice — da qualsiasi terminale:
delta

# Dalla root del progetto
cd ~/Desktop/DELTA-PLANT
.venv/bin/python main.py --preflight

# Avvio diretto (auto-rilancio venv se necessario)
python3 main.py

# Con venv esplicito
source .venv/bin/activate && python main.py
```

**COMANDO GLOBALE 'delta'**

Il comando 'delta' è installato in /usr/local/bin/delta e richiama automaticamente AVVIO_DELTA.py dalla cartella del progetto. Non è necessario navigare nella directory del progetto né attivare manualmente il venv prima dell'avvio.

## 5.2 Menu CLI

All'avvio viene mostrato il menu interattivo. Le opzioni disponibili sono:

| | |
|---|---|
| **[1] Avvia diagnosi pianta** | Acquisisce immagine + dati sensori → diagnosi completa |
| **[3] Dati sensori correnti** | Mostra una lettura istantanea di tutti i sensori |
| **[4] Esporta dati in Excel** | Genera/aggiorna il file exports/delta_diagnoses.xlsx |
| **[5] Ultime diagnosi** | Mostra le diagnosi recenti memorizzate nel database |
| **[6] DELTA Academy** | Modulo di formazione interattiva per l'operatore |
| **[7] Pannello Amministratore** | Funzioni avanzate protette da password |
| **[8] Cartella immagini input** | Visualizza e gestisce la cartella input_images/ (no-camera) |
| **[C] Chat domanda/risposta libera** | Interagisci liberamente con l'orchestrator DELTA: scrivi una domanda o comando, ricevi risposta immediata. Digita '/exit' per tornare al menu. |
| **[0] Esci** | Spegne il sistema in modo sicuro |

## Guida all'uso: Chat domanda/risposta libera (CLI)

Selezionando l'opzione [C] dal menu CLI, si accede alla modalità chat libera con l'orchestrator DELTA. L'utente può scrivere qualsiasi domanda, comando o richiesta di spiegazione. Il sistema risponde in tempo reale sfruttando il motore di orchestrazione integrato. Per uscire dalla chat e tornare al menu principale, digitare '/exit'.

- Esempi: 'Qual è lo stato attuale della pianta?', 'Mostrami le ultime diagnosi', 'Spiega il rischio attuale'
- La chat libera è utile per domande rapide, troubleshooting e interazione naturale con il sistema.

Copyright © 2026 Paolo Ciccolella. All rights reserved.

La funzione è disponibile solo se l'orchestrator DELTA è correttamente installato e attivo.

- Digitare '/exit' per uscire dalla chat e tornare al menu principale.

## 5.3 Raccolta dati sensori

Il sistema avvia automaticamente un thread in background che legge i sensori ogni 30 secondi. Se l'hardware non è collegato, genera dati simulati per sviluppo e test.

Modalità operative del SensorReader:

- hardware — lettura reale via I2C (richiede sensori fisici collegati)
- simulated — valori casuali realistici per test e sviluppo
- manual — inserimento interattivo da tastiera (CLI opzione dedicata)

## 5.4 Diagnosi pianta — flusso operativo v3.2

- 1. Acquisizione frame dalla Pi Camera (o immagine da cartella input_images/)
- 2. Rilevamento organo: foglia (modalita leaf-only v3.0 attiva di default)
- 3. Pre-processing: ridimensionamento a 224×224, normalizzazione
- 4. Segmentazione foglia tramite filtro HSV verde (o GrabCut) e selezione backend attivo
- 5. Inferenza AI: top-1, top-3, confidenza, fallback Classe_NonClassificato e spiegazione opzionale
- 6. Lettura sensori ambientali (temperatura, umidita, luce, CO2, pH, EC) o input manuale guidato
- 7. Applicazione 21 regole agronomiche -> attivazione regole + livello rischio
- 8. Oracolo Quantistico di Grover -> Quantum Risk Score [0,1]
- 9. Revisione contestuale LLM e Q&A follow-up se la diagnosi richiede approfondimento
- 10. Generazione raccomandazioni pratiche per l'operatore
- 11. Output Telegram/CLI/API con overlay LayerCAM quando la explainability e disponibile; in v3.2 puo essere generata da EfficientFormer anche se la classificazione attiva resta sul backend generale
- 12. Salvataggio in SQLite, Excel, memoria diagnostica persistente e log runtime

## 5.4a Diagnosi intelligente Telegram — flusso v3.2

In v3.2, il flusso diagnosi su Telegram include una fase di raccolta della descrizione utente e, in caso di pianta non riconosciuta, un ciclo di domande di approfondimento conversazionale con l'LLM. Il flusso si attiva sia dal menu (pulsante Diagnosi) sia inviando liberamente una foto in chat senza passare dal menu.

- 1. Ricezione foto (da menu Diagnosi o invio libero in chat)
- 2. DELTA chiede: 'Di che pianta si tratta? Descrivi l'anomalia riscontrata'
- 3. L'utente descrive la pianta e il problema (es. 'Pomodoro con macchie gialle sulle foglie')
- 4. Scelta sensori: automatici o inserimento manuale passo-passo
- 5. Inferenza AI sul modello TFLite (33 classi PlantVillage)
- - Se confidenza >= 50%: ramo normale (punto 6)
- - Se confidenza < 50%: ramo fallback conversazionale (punto 7)
- 6. Ramo normale — reclassificazione con LLM:
- • L'LLM verifica se la classe PlantVillage rilevata e coerente con la descrizione utente
- • Se non lo e, corregge la classe con quella piu appropriata tra le 33 disponibili
- • Diagnosi strutturata generata con descrizione utente integrata nel prompt
- 7. Ramo fallback — follow-up conversazionale:
- • Avviso: 'Pianta non riconosciuta nel database'
- • L'LLM genera fino a 5 domande mirate (colore, parte colpita, progressione, ecc.)

Copyright © 2026 Paolo Ciccolella. All rights reserved.

• Stop anticipato se l'LLM ritiene di avere gia informazioni sufficienti

- • Diagnosi finale strutturata in 5 blocchi: ipotesi, differenziale, azioni immediate,
- azioni a breve termine, monitoraggio — con esplicito avviso di incertezza
- 8. Risposta inviata in chat con paginazione automatica (/continua se testo lungo)
- 9. Voce contestuale: nelle fasi testuali della diagnosi guidata DELTA accetta messaggi vocali, li trascrive e li inoltra allo stesso handler usato per il testo
- - Fasi supportate: descrizione pianta/anomalia, inserimento manuale sensori, risposte di follow-up
- - Fasi non supportate: attesa foto, scelta sorgente immagine e pulsanti inline obbligatori

| | |
|---|---|
| **Soglia fallback** | confidenza < 50% sul modello TFLite reale (configurabile in core/config.py) |
| **Max domande follow-up** | 5 (configurabile tramite MAX_FOLLOWUP_QUESTIONS in telegram_bot.py) |
| **Reclassificazione LLM** | Attiva solo se l'utente ha fornito una descrizione |
| **Diagnosi conversazionale** | Basata su descrizione + Q&A + dati sensori, senza immagine classificata |
| **Foto libera** | Stessa procedura del flusso guidato: foto → descrizione → sensori → diagnosi |
| **Voce in diagnosi guidata** | Supportata nelle sole fasi testuali (descrizione, sensori manuali, follow-up); bloccata quando DELTA attende foto o pulsanti |
| **Diagnosi singolo msg** | v3.2 — la diagnosi è inviata in un solo messaggio (split automatico a 4000 char) |
| **Chiusura conversazione** | v3.2 — dopo 'Posso fare qualcos'altro per te?' il bot resta in attesa, nessun messaggio non sollecitato |
| **HF max_tokens** | 1500 (override via env HF_MAX_TOKENS); timeout 60s (HF_TIMEOUT) |

## 5.4b Chat libera Telegram — memoria persistente

La chat libera Telegram, la modalità /chat e i flussi diagnostici che richiedono spiegazioni LLM usano ora lo stesso motore conversazionale condiviso. Questo elimina disallineamenti di contesto tra domanda libera, approfondimento della diagnosi e richieste successive dell'utente. In più, DELTA salva localmente anche il referto diagnostico strutturato più recente, così le richieste di follow-up restano ancorate alla diagnosi effettivamente prodotta.

- Memoria persistente per utente: gli ultimi 20 turni sono salvati in memory/sessions/<user_id>.json e vengono ricaricati dopo ogni riavvio del bot.
- Coerenza del contesto: chat libera, /chat e diagnosi Telegram leggono e scrivono la stessa cronologia conversazionale.
- Isolamento dei flussi: la chat libera globale viene sospesa automaticamente durante diagnosi guidata, inserimento manuale sensori, upload immagine e sessione /chat dedicata, evitando risposte spurie ai valori numerici dei sensori o doppie risposte LLM.
- Memoria diagnostica strutturata: dopo ogni diagnosi DELTA salva anche nome pianta, stato, classe AI, analisi organi, rischio, QRS, raccomandazioni, sensori e anomalie, per permettere approfondimenti mirati su ogni elemento del referto.
- Ri-ancoraggio automatico: se l'utente scrive richieste ellittiche come 'approfondisci', 'spiega il rischio' o 'qual è il nome della pianta', il motore conversazionale riusa esplicitamente l'ultima diagnosi come contesto principale.
- Igiene della memoria: i messaggi di errore del backend LLM non vengono salvati nella cronologia, per evitare contaminazioni del contesto.
- Ripresa automatica: se l'invio della diagnosi fallisce lato Telegram, i flag interni di diagnosi vengono comunque ripuliti e la chat libera torna disponibile.
- Rientro da NASA/SAR: dopo il dashboard generato via Mini App Telegram, DELTA chiude automaticamente lo stato

Copyright © 2026 Paolo Ciccolella. All rights reserved.

di attesa GPS e il follow-up puo continuare subito in chat libera, anche con messaggi vocali.

- Uso operativo: dopo una diagnosi l'utente può chiedere 'approfondisci', 'spiegami il rischio', 'qual è il nome della pianta' o 'spiegami la raccomandazione 2' senza perdere il contesto recente.

**[!] ATTENZIONE**

I file memory/sessions/*.json contengono cronologia conversazionale locale e sono dati runtime del dispositivo. Vanno mantenuti sul Raspberry o sul PC locale e non devono essere pubblicati su GitHub.

## 5.4c Pannello amministratore — statistiche bot

Dal pannello amministratore (interface/admin.py) la voce [10] 'Statistiche inferenza DELTAPLANO_bot' produce un report per singolo utente autorizzato con: numero richieste LLM, errori, lunghezza media risposta, modello usato e ultimo accesso. Include un istogramma ASCII comparativo tra tutti gli utenti autorizzati e una sezione 'Altri utenti' per quelli non in scientists.json. Legge anche file di log ruotati (.1 e .gz), deduplica righe duplicate e ignora tracce sintetiche di test (fake-model/FakeEngine), evitando conteggi gonfiati. Il report viene salvato in logs/deltaplano_inference_stats.txt.

## 5.4d Pannello amministratore — monitor visitatori del sito

Dal pannello amministratore la voce [11] 'Monitor visitatori deltaplant.ai' mostra il traffico del sito pubblico in forma first-party e privacy-safe. Gli eventi vengono registrati solo dopo consenso analytics esplicito. Il monitor interroga prima il report live protetto dell'API pubblica e, solo se non disponibile, ricade sul database SQLite locale configurato, senza memorizzare identificativi personali in chiaro. Il monitor consente di vedere panoramiche a 24 ore, 7 giorni e 30 giorni, pagine piu viste, referrer, campagne UTM, dispositivi, lingue, eventi recenti, diagnostica del report remoto ed export JSON del report.

- Eventi raccolti con consenso: page view, session start, uscita pagina con engagement, click outbound, avvio dashboard NASA/SAR, submit riuscito del form contatti, cambio lingua e aggiornamenti consenso.
- Storage operativo: gli eventi risiedono nello storage runtime dell'istanza che esegue l'API DELTA; il percorso predefinito e data/analytics/website_events.sqlite ma puo essere ridefinito dalla configurazione. Nessun invio a piattaforme advertising o pixel terzi.
- Report remoto protetto: il pannello admin firma un bearer JWT con scope admin:analytics:read verso /api/analytics/report; se la chiamata fallisce, il monitor mostra il motivo del fallback e puo aprire la voce [6] Diagnostica report remoto.
- Autorita del consenso: l'endpoint /api/analytics/events accetta eventi solo se la categoria analytics risulta abilitata lato server nella registry cifrata del consenso.
- Bootstrap consenso: dopo redeploy o reset dello storage runtime, il frontend risincronizza il consenso analytics lato server prima di inviare nuovi eventi.
- Operativita sul sito: su viewport larghi il consenso puo comparire come banner Privacy & cookies; su viewport compatti o mobile puo apparire invece il pulsante Privacy che apre lo stesso Preference Center. In entrambi i casi e necessario abilitare analytics per alimentare il monitor.
- Verifica pratica: dopo l'accettazione analytics dalla landing live, il report remoto puo mostrare subito eventi come privacy_update, session_start e page_view senza dover riavviare il backend o l'admin panel.
- Uso operativo: il monitor e pensato per il controllo interno dell'efficacia del sito e per verificare quali pagine o call-to-action generano piu interesse, senza pubblicare i dati raccolti.

## 5.5 Output diagnosi — interpretazione

| | |
|---|---|
| **Stato pianta** | Classe fitosanitaria rilevata dal modello AI |

Copyright © 2026 Paolo Ciccolella. All rights reserved.

| | |
|---|---|
| **Confidenza** | Percentuale di certezza della classificazione |
| **Simulato** | Sì se il modello .tflite non è disponibile (output casuale bias-Sano) |
| **Rischio globale** | nessuno / basso / medio / alto / critico |
| **QRS Grover** | Quantum Risk Score [0,1] + livello calcolato dall'Oracolo |
| **Revisione umana** | Richiesta se confidenza < 50% su modello REALE (non simulato) |
| **Regole attivate** | Lista delle soglie agronomiche violate |
| **Raccomandazioni** | Azioni pratiche suggerite all'operatore |

Codice colore rischio nel terminale:

- Verde — nessun rischio rilevato
- Ciano — rischio basso, monitorare
- Giallo — rischio medio, intervenire a breve
- Rosso chiaro — rischio alto, intervenire subito
- Rosso bold — rischio critico, intervento urgente

## 5.5a Modalità simulazione — comportamento senza modello

Quando il file models/plant_disease_model_39classes.tflite non è presente o il modello non è caricabile (es. runtime TensorFlow/TFLite assente), il sistema opera in modalità simulazione senza bloccare l'avvio.

| | |
|---|---|
| **Log ai livello** | WARNING al primo evento runtime non disponibile, poi DEBUG sulle inferenze simulate (evita spam di log mantenendo tracciabilità). |
| **Confidenza bassa su simulato** | INFO — 'Confidenza bassa su output simulato — nessun modello reale.' Non viene richiesta revisione umana urgente in modalità simulazione. |
| **WARNING confidenza bassa** | Emesso SOLO quando il modello reale è caricato e la confidenza è < 50%. In questo caso la revisione umana è genuinamente necessaria. |
| **Flag 'Simulato: Sì' nell'Excel** | Tutte le diagnosi simulate sono marcate nella colonna 'Simulato' del file .xlsx per distinguerle dalle diagnosi reali nelle analisi successive. |

**QUANDO INSTALLARE IL MODELLO REALE**

La modalità simulazione è adatta a sviluppo, formazione (DELTA Academy) e test. Per diagnosi fitosanitarie operative è necessario copiare il file plant_disease_model_39classes.tflite nella cartella models/. Con runtime AI assente l'avvio resta disponibile in modalità degradata, ma il preflight AI continuerà a fallire finché non si installa ai-edge-litert==1.2.0 (RPi5 aarch64) o tensorflow (desktop/x86).

## 5.6 Inserimento manuale dati

Se i sensori fisici non sono disponibili, il sistema accetta inserimento manuale da tastiera. Alla voce [3] del menu o durante una diagnosi in modalità manuale, il sistema richiede i valori uno per uno. Premere INVIO senza digitare per saltare un campo (valore = null).

## 5.7 Export Excel

L'export Excel genera il file operativo con una finestra mobile degli ultimi 30 giorni. Ogni nuova diagnosi viene inclusa nel report e i record piu vecchi di 30 giorni vengono rimossi automaticamente dal database prima dell'esportazione. Le colonne includono tutti i parametri ambientali, i risultati AI e le raccomandazioni.

```
> TERMINALE
# File generato in:
/home/proctor81/Desktop/DELTA-PLANT/exports/delta_diagnoses.xlsx
```

Copyright © 2026 Paolo Ciccolella. All rights reserved.

## 5.8 Pipeline X e aggiornamento del manuale — v3.2

Pipeline X e il flusso MLOps resumable introdotto per orchestrare l'intera catena EfficientFormer: fine-tuning, export, evaluation, benchmark, materiali divulgativi e, da questa release, anche la rigenerazione del manuale PDF. Lo stato viene salvato in models/pipeline_x_state.json, quindi il comando --resume riparte sempre dal primo step mancante senza rilanciare inutilmente il training.

| | |
|---|---|
| **State file** | models/pipeline_x_state.json |
| **Stop cooperativo** | models/pipeline_x.stop |
| **Step finali** | evaluate -> benchmark -> dissemination -> manual |
| **Output manuale** | Manuale/DELTA_Manuale_Utente.pdf |

> PIPELINE X

```
# Riprende la pipeline dal primo step mancante
python tools/pipeline_x.py --resume

# Artefatti aggiornati a fine pipeline
logs/vision_eval/comparison_summary.json
logs/vision_benchmark.json
logs/attivita_divulgative/dissemination_summary.json
Manuale/DELTA_Manuale_Utente.pdf
```

Copyright © 2026 Paolo Ciccolella. All rights reserved.

# 6. PARTE B (cont.) — API REST E TELEGRAM

## 6.1 API REST Flask (opzionale)

L'API è attualmente ABILITATA (impostazione enable_api in core/config.py). Se abilitata, il server è raggiungibile su http://0.0.0.0:5000

Per abilitarla modificare core/config.py:

```
> TERMINALE
API_CONFIG = {
    "enable_api": True,   # <-- cambiare in True
    "host": "0.0.0.0",
    "port": 5000,
}
```

Endpoint principali disponibili:

| | |
|---|---|
| **GET /health** | Stato del sistema: modello pronto, sensori hw, record DB |
| **POST /diagnose** | Avvia una nuova diagnosi. Body JSON opzionale: { sensor_data: {...} } |
| **GET /sensors** | Ultimi dati sensori acquisiti dal thread in background |
| **GET /sensors/read** | Forza una nuova lettura istantanea dei sensori |
| **GET /diagnoses** | Lista ultime N diagnosi. Query param: ?limit=50 (max 500) |
| **GET /diagnoses/<id>** | Singolo record per ID numerico |
| **GET /model/info** | Informazioni sul modello AI: etichette, shape input, backend |

## 6.2 Bot Telegram (opzionale)

DELTA integra un bot Telegram per l'utilizzo conversazionale. Il bot e attualmente ABILITATO in core/config.py.

Il bot richiede l'API REST attiva e un token Telegram valido. L'API usata dal bot e: http://localhost:5000.

```
> TERMINALE
# Metodo consigliato: file .env nella directory di progetto
cp .env.example .env
# Modifica .env e inserisci: DELTA_TELEGRAM_TOKEN=<token>

# Oppure variabile d'ambiente (sesssione corrente)
export DELTA_TELEGRAM_TOKEN="TOKEN_BOT"

# Avvio rapido interattivo con API + Telegram
python main.py --enable-api --enable-telegram

# Avvio persistente/daemon (consigliato per servizio e test runtime)
python main.py --enable-api --enable-telegram --daemon
```

Parametri principali di configurazione:

| | |
|---|---|
| **enable_telegram** | True |
| **token_env** | DELTA_TELEGRAM_TOKEN |
| **api_base_url** | http://localhost:5000 |
| **authorized_usernames_file** | /home/proctor81/Desktop/DELTA-PLANT/data/telegram_scientists.local.json |

Copyright © 2026 Paolo Ciccolella. All rights reserved.

| | |
|---|---|
| **authorized_usernames** | [] |
| **authorized_users** | [] |
| **request_timeout_sec** | 5 |
| **conversation_timeout_sec** | 300 |
| **poll_interval_sec** | 1.0 |

Gli utenti autorizzati possono essere gestiti dal Pannello Amministratore nella sezione 'Scientists Telegram'. La lista e salvata in /home/proctor81/Desktop/DELTA-PLANT/data/telegram_scientists.local.json (nickname con @, file locale non tracciato dal repository).

- Comandi principali: /menu, /diagnosi, /upload, /images, /report, /dettaglio <id>, /sensori
- /export, /preflight, /academy, /license, /health, /batch, /nasa_sar
- Chat dedicata: pulsante inline 'Chiedi a DELTA Plant' e comando /chat per aprire una sessione esplicita di consulenza LLM.
- Chat libera: puoi scrivere direttamente in chat una domanda o richiesta (es. 'Mostrami lo stato', 'Spiega il rischio') e ricevere risposta dall'orchestrator DELTA.
- Lingua operativa: il messaggio /start e sempre presentato in italiano; la selezione Ryan (EN) o Cris (IT) pilota invece sia la voce TTS sia la lingua delle risposte in chat libera.
- Modalita voce: /voice on | off | auto controlla se DELTAPLANO deve rispondere anche in vocale; /voice_lang it | en | auto forza il pack voce da usare in chat libera.
- Modalita NASA-ISRO/SAR: pulsante inline 'Connettiti a NASA-ISRO/SAR' o comando /nasa_sar per avviare acquisizione GPS automatica via Mini App Telegram oppure invio manuale della posizione.
- Diagnosi spiegabile: il bot puo inviare foto originale, overlay LayerCAM e spiegazione testuale nello stesso flusso diagnostico; l'overlay puo essere generato da EfficientFormer anche se la classificazione attiva resta sul backend generale.
- Se il bot non risponde, verificare token, autorizzazioni e API attiva

## Guida all'uso: Chat libera in DELTAPLANO (Telegram)

Oltre ai comandi strutturati, il bot DELTAPLANO permette di interagire liberamente in chat: l'operatore può scrivere una domanda o richiesta direttamente nella conversazione Telegram. Il sistema DELTA risponde in modo contestuale sfruttando l'orchestrator AI. Questa modalità è ideale per domande rapide, richieste di spiegazione o troubleshooting senza dover seguire un flusso guidato.

- Esempi: 'Qual è la situazione attuale?', 'Spiega la diagnosi', 'Mostra le ultime raccomandazioni', 'Come si aggiorna il modello?'
- La chat libera è disponibile solo se l'orchestrator DELTA è attivo e il bot Telegram è correttamente configurato.
- Voce in chat libera: i messaggi vocali vengono trascritti; in modalita auto testo→testo e voce→voce, mentre /voice on e /voice off permettono di forzare il comportamento.
- Durante diagnosi guidata, inserimento manuale dei sensori, upload o sessione /chat dedicata, la chat libera viene sospesa automaticamente per evitare interferenze tra i flussi.
- Eccezione guidata: nella diagnosi Telegram i vocali sono comunque ammessi nelle sole fasi testuali (descrizione, sensori manuali, follow-up), ma non durante attesa foto o pulsanti obbligatori.
- Per tornare ai comandi guidati, usa /menu o i pulsanti inline.

### 6.2.1 Ambiente DELTAPLANO (Telegram)

DELTAPLANO e l'ambiente operativo in cui l'operatore interagisce con DELTA attraverso la piattaforma Telegram.

Copyright © 2026 Paolo Ciccolella. All rights reserved.

Consente un accesso rapido e guidato alle funzioni principali senza utilizzare direttamente la CLI locale.

## Funzionalita principali

- Diagnosi intelligente con foto da smartphone o immagini in input_images
- - Descrizione utente raccolta prima dell'inferenza AI
- - Reclassificazione LLM se la classe non coincide con la descrizione
- - Follow-up conversazionale (max 5 domande) se la pianta non e nel database
- Invio foto libero in chat: stessa procedura guidata della diagnosi da menu
- Upload immagini per acquisizione e diagnosi da archivio input_images
- Report, export Excel e storico diagnosi direttamente in chat
- Academy, preflight AI e dati sensori disponibili via Telegram
- Interfaccia con pulsanti inline per ridurre errori di input

## Installazione e attivazione

Per abilitare DELTAPLANO e necessario configurare il bot Telegram e fornire il token nel file .env. L'API REST deve essere attiva. Su Raspberry Pi il servizio systemd gestisce l'avvio automatico del bot ad ogni accensione del dispositivo.

> TERMINALE

```
# 1. Crea il bot con @BotFather su Telegram
# 2. Copia il file template e inserisci il token
cp .env.example .env
#    Imposta: DELTA_TELEGRAM_TOKEN=<token_da_BotFather>

# 3. (Raspberry Pi) Installa l'autostart
sudo bash install_service.sh

# 4. (PC/sviluppo) Avvio manuale con API + Telegram
python main.py --enable-api --enable-telegram

# 5. (Runtime persistente) Avvio daemon
python main.py --enable-api --enable-telegram --daemon
```

## Verifica operativa al boot (Raspberry Pi)

Dopo l'installazione del servizio, DELTA e DELTAPLANO devono risultare attivi automaticamente a ogni riavvio. Se il servizio risulta abilitato (enabled) ma non attivo (inactive), controllare immediatamente i log del servizio e lo stato della rete.

> TERMINALE

```
# Stato del servizio DELTA
systemctl is-enabled delta   # atteso: enabled
systemctl is-active delta    # atteso: active

# Riavvio manuale del servizio
sudo systemctl restart delta

# Log in tempo reale (diagnostica Telegram/API/CLI)
journalctl -u delta -f
```

Copyright © 2026 Paolo Ciccolella. All rights reserved.

**CHECK RAPIDO DELTAPLANO**

Con token valido in .env e rete disponibile, il bot Telegram risponde automaticamente dopo il boot del Raspberry Pi. In caso di mancata risposta verificare: token, stato del servizio delta e connettivita internet.

## Praticita operativa

DELTAPLANO e ideale per la raccolta dati sul campo e la reportistica rapida: l'operatore puo avviare diagnosi, leggere report e condividere risultati direttamente da smartphone o PC, con un flusso guidato e tracciabile.

## Flusso NASA-ISRO/SAR via Telegram

Il flusso NASA-ISRO/SAR e disponibile direttamente dal menu principale di DELTAPLANO e restituisce una risposta immediata al click del pulsante. Il bot porta poi l'utente alla raccolta della posizione GPS e genera una dashboard sintetica con mappa, localita, indicatori climatici live, tabella giornaliera compatta e contesto di follow-up in chat.

| | |
|---|---|
| **Entry point** | Pulsante inline 'Connettiti a NASA-ISRO/SAR' oppure comando /nasa_sar |
| **Acquisizione GPS** | Mini App Telegram 'Apri NASA-ISRO/SAR GPS' o bottone 'Invia GPS manualmente' |
| **Area analizzata** | Cerchio di 50 m centrato sul punto GPS condiviso dall'utente |
| **Output restituiti** | Snapshot mappa, report grafico NASA-ISRO/SAR, dashboard testuale live e contesto per domande successive |
| **Follow-up post-dashboard** | Al termine del dashboard la chat torna subito disponibile per domande testuali o vocali di approfondimento, senza dover usare /menu |
| **Fonti dati operative** | Open-Meteo Archive per temperatura/umidita/pioggia giornaliere e NASA POWER (GWETROOT/GWETTOP) per proxy umidita del suolo |
| **Fallback operativi** | Se la Web App non restituisce coordinate valide, il bot invita all'invio manuale della posizione |
| **Operativita post-fix** | Il click del menu produce subito l'istruzione iniziale; dopo aggiornamenti del bot e richiesto restart del servizio delta |

**> FLUSSO NASA-ISRO/SAR TELEGRAM**

```
# Flusso operativo NASA-ISRO/SAR da Telegram
1. /menu
2. Tap su 'Connettiti a NASA-ISRO/SAR'
3. Il bot risponde con il prompt operativo e mostra due opzioni:
  - 'Apri NASA-ISRO/SAR GPS' (Mini App Telegram)
  - 'Invia GPS manualmente'
4. Dopo il GPS, DELTA invia:
  - mappa dell'area di interesse
  - report grafico NASA-ISRO/SAR
  - dashboard testuale live con riga iniziale 'Oggi', Daily breakdown giornaliero e rischio settimanale
5. L'utente puo continuare con domande di approfondimento nella stessa chat
```

- La riga iniziale del report usa sempre il giorno piu recente disponibile nella finestra di 7 giorni, quindi i valori mostrati in apertura corrispondono al giorno corrente della serie e non al primo giorno storico della tabella.
- La sezione 'Daily breakdown' e ordinata dal giorno piu recente al piu vecchio e usa colonne compatte (Data, Soil%, TempC, RH%, Rain, Risk%) per restare leggibile anche dallo schermo di uno smartphone.
- Il rischio fungino giornaliero usa la stessa formula del modulo operativo: humidity_factor = clamp((RH - 60) / 40, 0, 1),

Copyright © 2026 Paolo Ciccolella. All rights reserved.

temp_factor = clamp((T - 12) / 18, 0, 1), rain_factor = clamp(pioggia_mm / 14, 0, 1), persistenza = rischio_precedente * 0.05, quindi score = clamp(0.40 * humidity_factor + 0.35 * temp_factor + 0.25 * rain_factor + persistenza, 0, 1).

- Il rischio fungino settimanale riportato nel dashboard e la media aritmetica dei 7 valori giornalieri; l'etichetta verbale e classificata in BASSO, MEDIO o ALTO sulla stessa logica usata dalla landing Raspberry Pi.
- Se NASA POWER restituisce valori mancanti o sentinella (-999) sugli ultimissimi giorni, DELTA mantiene l'ultimo proxy di umidita del suolo valido allineato alle date meteo invece di generare una settimana sintetica tutta uguale.
- Il ritorno del payload Mini App ripulisce lo stato interno di attesa GPS; questo evita che il primo follow-up vocale venga scambiato per una diagnosi guidata ancora aperta.
- Se il client Telegram non supporta la Mini App o il consenso GPS viene negato, usare l'invio manuale della posizione.
- Il bottone del menu e il comando /nasa_sar portano allo stesso flusso applicativo.
- Dopo modifiche al bot Telegram, applicare un riavvio del servizio con 'sudo systemctl restart delta' per rendere visibili i nuovi comportamenti.
- Se il restart non sostituisce il PID del processo oppure va in timeout, verificare 'systemctl status delta' e forzare un ciclo pulito con 'sudo systemctl kill -s SIGKILL delta && sudo systemctl start delta'.
- Per diagnostica rapida: 'journalctl -u delta -n 50 --no-pager' oppure 'journalctl -u delta -f'.

## Flusso NASA-ISRO/SAR sul sito pubblico deltaplant.ai

La landing pubblica deltaplant.ai espone una demo NASA-ISRO/SAR live allineata alla stessa logica operativa della dashboard Telegram. Quando il backend api.deltaplant.ai e raggiungibile, la pagina usa lo stesso payload di analisi e restituisce gli stessi indicatori chiave per la lettura rapida del rischio agronomico.

| | |
|---|---|
| **Input area** | Coordinate GPS dell'area selezionata e buffer operativo di 50 m, coerente con il flusso Telegram |
| **Pipeline dati** | Open-Meteo Archive per meteo storico recente + NASA POWER per proxy umidita del suolo, con stesso ordinamento della dashboard Telegram |
| **Output principali** | Mappa area, localita, dashboard testuale live con riga Oggi, daily breakdown giornaliero e rischio settimanale sintetico |
| **Fallback web** | Se il backend live non e disponibile, la pagina resta utilizzabile in modalita demo locale senza bloccare la navigazione del sito |
| **Discovery web** | La landing pubblica espone metadati strutturati (Open Graph, Twitter Card, JSON-LD), canonical, sitemap.xml, robots.txt e llms.txt per facilitare indicizzazione su motori generalisti, discovery tecnologica/scientifica e retrieval su sistemi LLM/RAG |
| **Ottimizzazione mobile** | La DEMO NASA e stata compattata e verificata su viewport classe iPhone 14 Pro e Samsung Galaxy S22+, con input a 16 px, controlli touch >= 48 px, supporto safe-area e assenza di overflow orizzontale |
| **Privacy analytics** | Gli eventi del sito e della hero demo vengono inviati al monitor visitatori solo dopo consenso analytics esplicito; su mobile o viewport compatti l'accesso al consenso puo avvenire tramite pulsante Privacy invece del banner esteso. La demo NASA resta consultabile anche senza tracking |

- La landing e il bot Telegram condividono le stesse regole di lettura del rischio fungino e la stessa logica di presentazione del giorno piu recente disponibile.
- Le richieste live del sito passano dall'endpoint /api/nisar/nasa-only del backend pubblico, cosi la descrizione mostrata

Copyright © 2026 Paolo Ciccolella. All rights reserved.

sul web resta coerente con quella restituita in chat.

- Il file pubblico llms.txt fornisce a crawler semantici e pipeline RAG una sintesi canonica di DELTA Plant, DELTAPLANO, benchmark, URL preferiti e vincoli di interpretazione del dashboard NASA/SAR.
- Le pagine privacy, cookie e termini del sito espongono ora anch'esse metadati Open Graph, Twitter Card e JSON-LD coerenti con la landing, mentre la pagina di redirect privacy e marcata noindex per evitare duplicazione nei crawler.
- Il monitor visitatori first-party puo registrare anche l'avvio della hero dashboard, ma soltanto dopo consenso analytics valido lato server.
- Dopo l'abilitazione analytics dal Privacy Center della landing, il monitor remoto puo popolarsi immediatamente con privacy_update, session_start e page_view generati dalla stessa sessione web.

### Learning-by-Doing (upload + metadati)

Con il comando /upload (o inviando una foto direttamente in chat) l'immagine viene acquisita in input_images per analisi successive. Nella logica semplificata i passaggi runtime di etichettatura/training non sono attivi.

```
> TERMINALE
# Cartella learning-by-doing
/home/proctor81/Desktop/DELTA-PLANT/datasets/learning_by_doing/
  images/   # copie immagini per training
  records/  # metadati JSON per immagine
```

## 6.3 Training Offline (non runtime)

Il sistema runtime non include alcun riaddestramento on-device. Eventuali aggiornamenti del modello sono previsti come workflow offline tecnico (script in /ai e pipeline dedicate).

Procedura training offline consigliata:

- Raccogliere dataset nelle cartelle di training offline
- Eseguire script dedicati in /ai (train_keras_classifier.py, convert_keras_to_tflite.py)
- Validare artefatti con preflight prima del deploy
- Distribuire il nuovo .tflite in ambiente operativo e riavviare il servizio

Copyright © 2026 Paolo Ciccolella. All rights reserved.

# 7. DATABASE E PERSISTENZA

## 7.1 Schema database SQLite

DELTA utilizza un database SQLite per archiviare tutte le diagnosi. Il file è posizionato in: /home/proctor81/Desktop/DELTA-PLANT/delta.db
Record massimi prima della pulizia automatica: 10000
Retention temporale operativa: il sistema mantiene nel report e nel database solo gli ultimi 30 giorni di diagnosi.

Tabella principale: diagnoses

| | |
|---|---|
| **id** | Chiave primaria auto-incrementale |
| **timestamp** | Data/ora diagnosi (ISO 8601 UTC) |
| **temperature_c** | Temperatura in °C |
| **humidity_pct** | Umidità relativa in % |
| **pressure_hpa** | Pressione atmosferica in hPa |
| **light_lux** | Luminosità in lux |
| **co2_ppm** | CO2 in ppm |
| **ph** | pH del suolo |
| **ec_ms_cm** | Conducibilità elettrica in mS/cm |
| **sensor_source** | Fonte dati: hardware / simulated / manual |
| **ai_class** | Classe diagnostica rilevata |
| **ai_confidence** | Confidenza modello AI (0.0–1.0) |
| **plant_status** | Stato fitosanitario complessivo |
| **overall_risk** | Livello di rischio: nessuno/basso/medio/alto/critico |
| **needs_review** | Flag revisione umana richiesta (0/1) |
| **recommendations** | Raccomandazioni in formato testo |
| **created_at** | Timestamp inserimento record |

## 7.2 Backup del database

> TERMINALE

```
# Backup manuale
cp ~/DELTA/delta.db ~/DELTA/backup_delta_$(date +%Y%m%d).db

# Copia su USB
cp ~/DELTA/delta.db /media/pi/USB/delta_backup.db
```

## 7.3 Logging di sistema

| | |
|---|---|
| **File di log** | /home/proctor81/Desktop/DELTA-PLANT/logs/delta.log |
| **Livello** | DEBUG |
| **Dimensione max** | 10 MB |
| **File di backup** | 5 |

> TERMINALE

Copyright © 2026 Paolo Ciccolella. All rights reserved.

```
# Visualizza log in tempo reale
tail -f ~/DELTA/logs/delta.log

# Ultimi errori
grep ERROR ~/DELTA/logs/delta.log | tail -20
```

Copyright © 2026 Paolo Ciccolella. All rights reserved.

# 8. ARCHITETTURA SOFTWARE — MODULI

DELTA adotta un'architettura modulare: ogni sotto-directory corrisponde a un dominio funzionale autonomo. I moduli comunicano tramite la classe centrale DeltaAgent definita in core/agent.py.

## Package: Nucleo del sistema

| | |
|---|---|
| **core/agent.py** | DELTA - core/agent.py |
| **core/auth.py** | DELTA - core/auth.py |
| **core/config.py** | DELTA - core/config.py |

## Package: Lettura sensori

| | |
|---|---|
| **sensors/anomaly_detection.py** | DELTA - sensors/anomaly_detection.py |
| **sensors/filtering.py** | DELTA - sensors/filtering.py |
| **sensors/reader.py** | DELTA - sensors/reader.py |

## Package: Elaborazione immagini

| | |
|---|---|
| **vision/camera.py** | DELTA - vision/camera.py |
| **vision/efficientformer_classifier.py** | DELTA - vision/efficientformer_classifier.py |
| **vision/mobilenet_service.py** | DELTA - vision/mobilenet_service.py |
| **vision/organ_detector.py** | DELTA - vision/organ_detector.py |
| **vision/preprocessing.py** | DELTA - vision/preprocessing.py |
| **vision/segmentation.py** | DELTA - vision/segmentation.py |
| **vision/vision_backend.py** | DELTA - vision/vision_backend.py |
| **vision/vision_service.py** | DELTA - vision/vision_service.py |

## Package: Intelligenza artificiale

| | |
|---|---|
| **ai/convert_keras_to_tflite.py** | Conversione modello TensorFlow/Keras in TensorFlow Lite. |
| **ai/demo_hero_fallback.py** | (nessuna docstring) |
| **ai/download_dipladenia.py** | DELTA - ai/download_dipladenia.py |
| **ai/download_plantvillage.py** | DELTA - ai/download_plantvillage.py |
| **ai/download_plantvillage_33classes.py** | DELTA - ai/download_plantvillage_33classes.py |
| **ai/download_pretrained_model.py** | DELTA - ai/download_pretrained_model.py |
| **ai/evaluate_vision_backends.py** | Valutazione dettagliata su validation set PlantVillage per backend DELTA. |
| **ai/expiring_artifact_store.py** | (nessuna docstring) |
| **ai/export_efficientformer_tflite.py** | Fine-tuning + export pipeline per EfficientFormerV2-S1: |
| **ai/inference.py** | DELTA - ai/inference.py |
| **ai/model_loader.py** | DELTA - ai/model_loader.py |
| **ai/preflight_validator.py** | Validazione pre-avvio artefatti modello: modello, labels, immagine test. |
| **ai/quantum_risk.py** | DELTA - ai/quantum_risk.py |
| **ai/tflite_inference_runner.py** | Esecuzione inferenza TFLite su immagini di piante in modalità produzione. |
| **ai/train_keras_classifier.py** | Training classificatore immagini piante con TensorFlow/Keras. |
| **ai/voice_service.py** | (nessuna docstring) |

## Package: Motore di diagnosi

| | |
|---|---|
| **diagnosis/engine.py** | DELTA - diagnosis/engine.py |
| **diagnosis/rules.py** | DELTA - diagnosis/rules.py |

Copyright © 2026 Paolo Ciccolella. All rights reserved.

## Package: Raccomandazioni agronomiche

| | |
|---|---|
| **recommendations/agronomy_engine.py** | DELTA - recommendations/agronomy_engine.py |

## Package: Persistenza e export

| | |
|---|---|
| **data/database.py** | DELTA - data/database.py |
| **data/excel_export.py** | DELTA - data/excel_export.py |
| **data/logger.py** | DELTA - data/logger.py |

## Package: Interfaccia utente

| | |
|---|---|
| **interface/academy.py** | DELTA - interface/academy.py |
| **interface/admin.py** | DELTA - interface/admin.py |
| **interface/api.py** | DELTA - interface/api.py |
| **interface/cli.py** | DELTA - interface/cli.py |
| **interface/github_publisher.py** | DELTA - interface/github_publisher.py |
| **interface/telegram_bot.py** | (nessuna docstring) |

Copyright © 2026 Paolo Ciccolella. All rights reserved.

# 9. RISOLUZIONE PROBLEMI

### Problema: I sensori non vengono rilevati

- Verificare i collegamenti fisici SDA/SCL/VCC/GND
- Abilitare I2C: sudo raspi-config → Interface Options → I2C
- Scansionare il bus: sudo i2cdetect -y 1
- Confrontare gli indirizzi trovati con quelli in core/config.py

### Problema: La camera non funziona

- Verificare che il cavo flat sia inserito correttamente
- Abilitare la camera: sudo raspi-config → Interface Options → Camera
- Testare: libcamera-hello oppure libcamera-jpeg -o test.jpg
- Con OpenCV: python3 -c "import cv2; print(cv2.VideoCapture(0).isOpened())"
- Alternativa senza camera: usare la cartella input_images/ (menu [8] o [1]→S)

### [!] ATTENZIONE

Errore ImportError su moduli AI/sensori

### Problema: Export Excel non disponibile — [ERROR] openpyxl non installato

- Attivare l'ambiente virtuale: source .venv/bin/activate
- Installare il pacchetto mancante: pip install openpyxl
- Verifica: python3 -c "import openpyxl; print(openpyxl.__version__)"
- In alternativa, reinstallare tutte le dipendenze: pip install -r requirements.txt
- Nota: l'errore è non bloccante — la diagnosi continua, solo l'export .xlsx è disabilitato

### Problema: Runtime AI non disponibile / segfault su RPi5 (tensorflow/ai-edge-litert)

- Controllare la versione Python nel venv: python --version (usare Python 3.12)
- Su Raspberry Pi 5 (aarch64): pip install ai-edge-litert==1.2.0
- ATTENZIONE: ai-edge-litert >= 1.3.0 causa segfault su BCM2712 — usare solo la 1.2.0
- tflite-runtime non è disponibile su aarch64/Python 3.12 tramite pip
- Fallback desktop/x86: pip install tensorflow==2.21.0 flatbuffers==25.12.19
- Senza runtime, DELTA avvia in modalità simulata (non bloccante), ma preflight AI fallisce

### Problema: Inferenza AI lenta o non usa la NPU

- Verificare installazione AI HAT: hailortcli fw-control identify
- Assicurarsi che use_edge_tpu=True in core/config.py
- Verificare che il modello .tflite sia nella cartella models/
- Consultare la documentazione Hailo RT per i driver aggiornati

### [!] ATTENZIONE

Database SQLite corrotto

### Problema: Il manuale PDF non si aggiorna

- Eseguire: python Manuale/genera_manuale.py
- Verificare che fpdf2 sia installato: pip install fpdf2
- Verificare che core/config.py non abbia errori di sintassi

Copyright © 2026 Paolo Ciccolella. All rights reserved.

**Problema: Bot Telegram non risponde dopo il riavvio**

• Controllare servizio: systemctl is-enabled delta e systemctl is-active delta
• Se inactive: sudo systemctl restart delta
• Controllare i log: journalctl -u delta -n 50 --no-pager
• Verificare token in .env (DELTA_TELEGRAM_TOKEN valorizzato e non placeholder)
• Verificare connettivita di rete al boot (network-online target)
• Confermare che TELEGRAM_CONFIG['enable_telegram'] sia True in core/config.py

**Problema: Dopo il dashboard NASA/SAR un vocale viene rifiutato come diagnosi guidata**

• Comportamento atteso post-fix: dopo il dashboard NASA/SAR la chat libera torna subito disponibile e il follow-up puo arrivare anche in vocale.
• Riavviare il servizio dopo l'aggiornamento: sudo systemctl restart delta
• Verificare che il PID sia cambiato: systemctl show -p MainPID --value delta
• Controllare i log recenti: journalctl -u delta -n 80 --no-pager
• Se compare ancora il rifiuto, cercare tracce dello stato GPS pendente o del messaggio 'Voice mode is not supported during the guided DELTA Plant diagnosis'.
• Come workaround operativo immediato usare /menu una sola volta, poi riprovare il vocale.

**Problema: Il file plant_disease_model_39classes.tflite ha magic bytes GGUF (637 MB)**

• Sintomo: errore 'Model provided has model identifier \'' all'avvio — visione in modalità simulata.
• Causa: il file models/plant_disease_model_39classes.tflite e in realta un modello GGUF (llama.cpp) rinominato per errore.
• Verifica: python3 -c "with open('models/plant_disease_model_39classes.tflite','rb') as f: print(f.read(4))" — se stampa b'GGUF' e il problema.
• Fix 1 — Rinomina il GGUF: mv models/plant_disease_model_39classes.tflite models/plant_disease_model.gguf
• Fix 2 — Converti il Keras reale: python ai/convert_keras_to_tflite.py --keras-model models/plant_disease_model.keras --output models/plant_disease_model_39classes.tflite --quantization float16
• Fix 3 (alternativa rapida) — Usa il modello dipladenia: echo 'DELTA_ACTIVE_MODEL=dipladenia' >> .env
• Riavvia DELTA dopo il fix. Log atteso: '[INFO] delta.tflite.runner: Modello caricato con backend=ai_edge_litert'

**[!] ATTENZIONE**

La chat LLM non risponde o risponde con errore backend non disponibile

Copyright © 2026 Paolo Ciccolella. All rights reserved.

# 10. AGGIORNAMENTO DEL MANUALE

Questo manuale è generato automaticamente dal codice sorgente del progetto. Ogni volta che vengono apportate modifiche all'hardware o al software, è sufficiente rieseguire lo script di generazione per ottenere un PDF aggiornato.

## Quando rigenerare il manuale

- Dopo aver modificato soglie o parametri in core/config.py
- Dopo aver aggiunto/rimosso sensori o cambiato gli indirizzi I2C
- Dopo aver aggiunto nuovi moduli o funzionalità al software
- Dopo aver aggiornato requirements.txt con nuove dipendenze
- Prima di ogni rilascio o consegna documentazione

## Come rigenerare

> TERMINALE

```
cd ~/DELTA
source .venv/bin/activate

# Rigenera il PDF aggiornato
python Manuale/genera_manuale.py

# Il nuovo PDF sovrascrive il precedente in:
# Manuale/DELTA_Manuale_Utente.pdf
```

## Automazione con cron (opzionale)

Per aggiornare il manuale automaticamente a ogni modifica dei file sorgente, aggiungere un hook Git o uno script cron:

> TERMINALE

```
# Hook Git post-commit (.git/hooks/post-commit)
#!/bin/bash
cd $(git rev-parse --show-toplevel)
python Manuale/genera_manuale.py
git add Manuale/DELTA_Manuale_Utente.pdf
git commit --amend --no-edit
```

## Come estendere il manuale

Il file Manuale/genera_manuale.py è strutturato in funzioni indipendenti, una per sezione. Per aggiungere una nuova sezione:

- Creare una nuova funzione _add_mia_sezione(pdf, ...) nel file
- Aggiungere la sezione all'indice nella funzione main()
- Chiamare la funzione nella sequenza di build del PDF

Copyright © 2026 Paolo Ciccolella. All rights reserved.

# 11. DELTA ACADEMY — FORMAZIONE OPERATORE

DELTA Academy e il modulo di formazione interattiva integrato nel software. Permette all'operatore di apprendere l'utilizzo del sistema attraverso simulazioni realistiche, quiz teorici e tutorial guidati, senza necessita di hardware fisico collegato. L'output delle simulazioni e identico a quello reale, consentendo una formazione pratica ed efficace.

## 11.1 Accesso alla Academy

La DELTA Academy e accessibile direttamente dal menu principale del software. Selezionare l'opzione [6] DELTA Academy per accedere al menu di formazione.

```
> MENU PRINCIPALE
# Avviare DELTA normalmente
python DELTA.py

# Dal menu principale selezionare:
[6] DELTA Academy  <- Formazione operatore
```

## 11.2 Contenuti della Academy

| | |
|---|---|
| **[1] Tutorial Guidato** | Percorso passo-passo per il primo utilizzo: avvio, menu, diagnosi, interpretazione risultati e parametri ambientali chiave. Consigliato come primo accesso al sistema. |
| **[2] Sim. Identifica la Malattia** | Scenario clinico completo con sintomi visivi, dati sensori e output AI. L'operatore deve identificare la malattia tra 4 opzioni. +15 punti se corretto. |
| **[3] Sim. Valutazione del Rischio** | Dato il quadro clinico completo, l'operatore assegna il livello di rischio corretto (nessuno/basso/medio/alto/critico). Punteggio parziale se si sbaglia di un solo livello. +15 punti se corretto, +5 se vicinissimo. |
| **[4] Sim. Scegli l'Intervento** | Con diagnosi e rischio gia forniti, l'operatore sceglie le 2 azioni di intervento corrette tra 4 opzioni (2 corrette + 2 errate). +20 punti per entrambe corrette, +5 per una sola corretta. |
| **[5] Quiz Teorico** | 5 domande a risposta multipla su: parametri ambientali, funzionamento AI, protocolli I2C, interpretazione risultati, fine-tuning. +10 punti per risposta corretta. Le domande vengono estratte casualmente da un pool di 8. |
| **[6] Il Mio Progresso** | Riepilogo del punteggio totale, statistiche quiz e simulazioni, badge ottenuti e livello operatore (Principiante / Apprendista / Operatore / Esperto / Maestro). |

## 11.3 Scenari di Simulazione disponibili

La Academy include 5 scenari clinici completi, selezionati casualmente a ogni simulazione. Ogni scenario include: contesto colturale, descrizione sintomi visivi, tabella dati sensori con stato, output AI completo (classe, confidenza, top-3) e spiegazione didattica finale.

| | |
|---|---|
| **Oidio su pomodoro** | Infezione fungina da Erysiphe spp. — rischio ALTO |
| **Carenza di azoto su lattuga** | EC bassa + pH alto in idroponica — rischio MEDIO |
| **Marciume radicale** | Sovra-irrigazione con Pythium — rischio CRITICO |
| **Pianta in condizioni ottimali** | Scenario di riferimento sano — rischio NESSUNO |

Copyright © 2026 Paolo Ciccolella. All rights reserved.

| | |
|---|---|
| **Attacco di afidi** | Infestazione insetti + fumaggine — rischio ALTO |

## 11.4 Sistema di Progressione e Badge

Il progresso dell'operatore e salvato automaticamente nel file data/academy_progress.json e persiste tra le sessioni. Il sistema di livelli e badge incentiva la formazione continua.

| | | |
|---|---|---|
| **Livello Principiante** | 0 – 49 punti | — accesso iniziale |
| **Livello Apprendista** | 50 – 149 punti | — ha completato il tutorial |
| **Livello Operatore** | 150 – 299 punti | — operativita di base acquisita |
| **Livello Esperto** | 300 – 499 punti | — competenza avanzata |
| **Maestro DELTA** | 500+ punti | — padronanza completa del sistema |

Badge ottenibili:

- Primo Passo — 50 punti accumulati
- Studioso — 5 domande di quiz corrette
- Diagnosta — 3 simulazioni corrette
- Esperto in Formazione — 200 punti raggiunti
- Agronomo Digitale — 10 simulazioni corrette
- Maestro DELTA — 500 punti raggiunti

**FORMAZIONE PRIMA DELL'USO**

Si raccomanda di completare almeno il Tutorial Guidato e 2-3 simulazioni prima di operare con il sistema su colture reali. La DELTA Academy non richiede sensori o camera collegati: funziona completamente in autonomia.

Copyright © 2026 Paolo Ciccolella. All rights reserved.

# 12. ANALISI MULTI-ORGANO — FOGLIA, FIORE E FRUTTO

DELTA mantiene documentata in v3.2 anche l'analisi simultanea di tutti gli organi vegetali presenti nell'inquadratura: foglie, fiori e frutti. Il sistema rileva automaticamente quali organi sono presenti nell'immagine e applica modelli di diagnosi specifici per ciascuno, in modo trasparente per l'operatore.

## 12.1 Rilevamento automatico degli organi

Il modulo vision/organ_detector.py analizza ogni frame acquisito dalla camera e identifica la presenza di foglie, fiori e frutti tramite segmentazione HSV multi-range. Ogni organo viene valutato in modo indipendente e il risultato e incluso nel report diagnostico.

| | |
|---|---|
| **Foglia (verde)** | Range HSV: [25,40,40]–[85,255,255]. Rilevamento tramite area minima 500 px². Metodi disponibili: HSV (veloce) e GrabCut (preciso). |
| **Fiore (giallo/bianco/rosa/rosso)** | Combinazione di 5 range HSV. Rileva fiori di colori diversi: giallo (pomodoro), bianco (patata), rosa/viola (melanzana), rosso (papavero). |
| **Frutto (rosso/arancione/giallo/verde)** | Combinazione di 5 range HSV: rosso maturo (pomodoro, peperone), arancione (agrumi), giallo maturo (banana, limone), verde (uva, kiwi). |

## 12.2 Patologie del fiore diagnosticate

Il sistema include regole esperte e modello AI dedicati per l'analisi delle principali patologie floreali. La diagnosi viene attivata solo quando il fiore e rilevato con copertura sufficiente nell'immagine.

| | |
|---|---|
| **Caduta prematura fiore** | Triggere: umidita < 30% o temperature estreme durante fioritura. Causa: stress idrico, eccesso azoto, mancanza impollinazione. |
| **Aborto floreale** | Triggere: temperatura < 10°C o > 35°C. Causa: sbalzo termico, carenza di boro, stress multipli. |
| **Mancata allegagione** | Mancata trasformazione fiore in frutto. Azione: impollinazione manuale + boro fogliare 0.2–0.5%. |
| **Oidio del fiore** | Polvere bianca su petali. Triggere: umidita < 60% + alta T. Azione: zolfo micronizzato + riduzione umidita. |
| **Muffa grigia (Botrytis) del fiore** | CRITICO: umidita >= 80% durante fioritura. Azione immediata: fungicida iprodione + ventilazione. |
| **Bruciatura petali** | Eccesso luminoso > 80.000 lux. Azione: ombreggiatura 50%. |
| **Deformazione floreale** | Presenza acari eriofidi o tripidi. Azione: acaricida specifico. |

## 12.3 Patologie del frutto diagnosticate

Le patologie del frutto sono tra le piu critiche per la perdita economica diretta della produzione. DELTA diagnostica in tempo reale le principali anomalie con indicazioni operative immediate.

| | |
|---|---|
| **Marciume apicale** | Carenza calcio — frequente in pomodoro. pH > 6.8 + EC < 1.2. Azione: calcio nitrato fogliare 0.5-1% + irrigazione uniforme. |
| **Spaccatura frutto** | Stress idrico alternato (asciutto/umido). Azione: irrigazione costante + pacciamatura per umidita stabile. |
| **Scottatura solare** | Radiazione > 90.000 lux + temperatura > 35°C. Azione: ombreggiatura 30-40% |

Copyright © 2026 Paolo Ciccolella. All rights reserved.

| | |
|---|---|
| | + trattamento caolino. |
| **Muffa grigia frutto (Botrytis)** | CRITICO: fungicida fludioxonil + rimozione frutti infetti immediata. |
| **Alternariosi del frutto** | Fungicida tebuconazolo + miglioramento drenaggio. |
| **Rugginosita buccia** | Acari rust mite — zolfo o acaricida specifico. |
| **Carenza calcio frutto** | pH non ottimale o irrigazione irregolare. Calcio cloruro fogliare 0.5%. |

## 12.4 Regole aggiuntive nel sistema rule-based

L'aggiornamento ha aggiunto 9 nuove regole diagnostiche al sistema, per un totale di 21 regole attive (12 originali + 4 fiore + 5 frutto). Le regole sono valutate in parallelo ad ogni diagnosi.

| | |
|---|---|
| **FLOW_01 — Aborto floreale** | Temperatura < 10°C o > 35°C con fiore rilevato |
| **FLOW_02 — Caduta fiore** | Umidita < 30% con fiore rilevato |
| **FLOW_03 — Malattia fiore AI** | Classe AI fiore != Fiore_sano con alta confidenza |
| **FLOW_04 — Botrytis fiore** | Umidita >= 80% con fiore rilevato — CRITICO |
| **FRUT_01 — Marciume frutto** | Classe AI frutto: Marciume o Muffa con alta confidenza |
| **FRUT_02 — Spaccatura** | Classe AI frutto: Spaccatura con alta confidenza |
| **FRUT_03 — Carenza Ca frutto** | pH > 6.8 + EC < 1.2 con frutto rilevato |
| **FRUT_04 — Scottatura solare** | Luce > 80.000 lux + temperatura > 32°C con frutto |
| **FRUT_05 — Malattia frutto AI** | Classe AI frutto diversa da Frutto_sano |

**COLTURE CHE PRODUCONO FIORI E/O FRUTTI**

Il sistema attiva automaticamente l'analisi di fiore e frutto quando vengono rilevati nell'immagine. Non e necessario configurare manualmente il tipo di coltura. Esempi: pomodoro, peperone, melanzana, zucchino, cetriolo, fragola, vite, agrumi.

Copyright © 2026 Paolo Ciccolella. All rights reserved.

# 13. ORACOLO QUANTISTICO DI GROVER — QUANTIFICAZIONE DEL RISCHIO

DELTA integra in v3.2 una simulazione classica dell'Algoritmo di Grover per la quantificazione del rischio degli eventi avversi agronomici. L'oracolo analizza tutti i fattori di rischio attivi e produce un Quantum Risk Score (QRS) che misura la probabilita complessiva di un evento avverso grave, considerando le interazioni sinergiche tra fattori.

## 13.1 Principio dell'Algoritmo di Grover

L'Algoritmo di Grover e un algoritmo quantistico di ricerca non strutturata che offre un vantaggio quadratico rispetto agli algoritmi classici: O(sqrt(N)) invece di O(N) per trovare l'elemento cercato in N possibilita. In DELTA, questo si traduce nell'identificazione del rischio dominante tra 2^n possibili stati di rischio, dove n e il numero di qubit del registro.

| | |
|---|---|
| **Registro quantistico** | 4 qubit = 16 stati di rischio possibili. Ogni stato rappresenta un tipo di evento avverso (fungino, pH, EC, stress termico, ecc.). |
| **Superposizione iniziale** | Stato uniforme: amplitudine 1/sqrt(16) per ogni stato. Rappresenta massima incertezza iniziale sul rischio dominante. |
| **Oracolo U_f** | Inverte il segno delle ampiezze degli stati avversi attivi. Phase flip: U_f\|i> = -\|i> se i e uno stato avverso. |
| **Diffusore di Grover** | U_s = 2\|psi><psi\| - I. Amplifica le ampiezze degli stati marcati. Equivale a: new_state = 2 * mean(state) - state. |
| **Misura** | Distribuzione di probabilita dopo le iterazioni: P(i) = \|ampiezza(i)\|^2. Il QRS e la somma pesata P(avversi). |

## 13.2 Quantum Risk Score (QRS)

Il QRS e un valore normalizzato tra 0 e 1 che rappresenta il rischio quantistico complessivo dopo l'amplificazione di Grover. Formula: QRS = somma [ P_Grover(i) * w(i) ] per i negli stati avversi. Con bonus composto se >= 3 stati avversi interagiscono.

| | |
|---|---|
| **QRS < 0.25** | Rischio NESSUNO — condizioni normali |
| **0.25 <= QRS < 0.45** | Rischio BASSO — monitoraggio standard |
| **0.45 <= QRS < 0.65** | Rischio MEDIO — attenzione raccomandata |
| **0.65 <= QRS < 0.80** | Rischio ALTO — intervento entro 24 ore |
| **QRS >= 0.80** | Rischio CRITICO — intervento urgente immediato |

## 13.3 Mappa stati quantistici — regole diagnostiche

Ogni regola diagnostica e mappata su uno stato del registro quantistico. 16 stati totali (4 qubit), coprono tutte le categorie di rischio agronomico.

| | |
|---|---|
| **\|0000> Rischio fungino** | FUNG_01: alta umidita + temperatura |
| **\|0001> Carenza luminosa** | PHOTO_01: lux < soglia fotosintesi |
| **\|0010> Eccesso luminoso** | PHOTO_02: foto-inibizione |
| **\|0011> Carenza CO2** | CO2_01: < 400 ppm |
| **\|0100> Eccesso CO2** | CO2_02: > 3500 ppm |
| **\|0101> pH acido** | PH_01: < 6.0 |
| **\|0110> pH alcalino** | PH_02: > 7.0 |

Copyright © 2026 Paolo Ciccolella. All rights reserved.

| | |
|---|---|
| **\|0111> Tossicita salina** | EC_01: > 3.5 mS/cm — PESO 0.9 |
| **\|1000> Carenza nutrienti** | EC_02: < 0.8 mS/cm |
| **\|1001> Stress termico** | TEMP_01: T fuori 18-28°C |
| **\|1010> Malattia AI foglia** | AI_01: classe != Sano con alta confidenza |
| **\|1011> AI incerta** | AI_02: confidenza < soglia |
| **\|1100> Aborto floreale** | FLOW_01: T estrema con fiore — PESO 0.7 |
| **\|1101> Caduta fiore** | FLOW_02: umidita bassa con fiore |
| **\|1110> Marciume frutto** | FRUT_01: Botrytis/marciume frutto — PESO 0.85 |
| **\|1111> Spaccatura frutto** | FRUT_02: stress idrico frutto — PESO 0.65 |

## 13.4 Amplificazione di Grover e vantaggio

Il guadagno di amplificazione (Amplification Gain) misura quanto le probabilita degli stati avversi sono state amplificate rispetto alla distribuzione uniforme classica. Un gain di 4x significa che lo stato dominante ha probabilita 4 volte superiore rispetto a un approccio classico.

**> ESEMPIO OUTPUT CLI**

```
# Esempio output Oracolo Quantistico di Grover:
Oracolo Quantistico di Grover:
  QRS:          0.7234 [ALTO]
  Evento dom.:   Tossicita salina (EC elevata)
  Amplific.:     3.8x  |  Iterazioni Grover: 3
```

## 13.5 Integrazione nel flusso di diagnosi

L'oracolo viene eseguito automaticamente al termine di ogni diagnosi, dopo la valutazione delle regole esperte. Il QRS complementa (non sostituisce) il sistema rule-based: fornisce una misura quantistica del rischio composito che tiene conto delle interazioni sinergiche tra fattori.

- Il QRS e incluso nel riepilogo della diagnosi (campo quantum_risk)
- Le raccomandazioni urgenti vengono generate automaticamente per QRS >= 0.65
- Il file ai/quantum_risk.py contiene la simulazione completa
- Parametri configurabili in QUANTUM_CONFIG dentro core/config.py
- n_qubits, grover_iterations e soglie sono modificabili senza toccare il codice

**SIMULAZIONE CLASSICA — NON HARDWARE QUANTISTICO**

L'Oracolo di Grover in DELTA e una simulazione classica esatta eseguita su CPU tramite numpy. Non richiede hardware quantistico. La simulazione preserva la semantica quantistica (superposizione, interferenza, amplificazione) e produce risultati identici a quelli che si otterrebbero su un vero computer quantistico con 4 qubit. La complessita e O(k*N) con k iterazioni, dove N=16 stati — trascurabile su qualsiasi hardware.

Copyright © 2026 Paolo Ciccolella. All rights reserved.

# 14b. CHAT AI — HUGGINGFACE LLM E PULSANTE «CHIEDI A DELTA»

DELTA Plant integra un motore di chat AI basato su modelli linguistici di grandi dimensioni (LLM) via HuggingFace Inference API. L'utente può conversare liberamente con un esperto agronomico virtuale basato su Qwen3-14B, sia dal bot Telegram sia dalla CLI locale, senza bisogno di eseguire il modello in locale.

## 14b.1 Architettura

| | |
|---|---|
| **Modello primario** | Qwen/Qwen3-14B (HuggingFace InferenceClient) |
| **Fallback secondario** | Ollama locale (solo orchestrator, se configurato) |
| **Fallback finale** | Messaggio di errore controllato (nessun backend disponibile) |
| **Modulo Python** | llm/huggingface_llm.py — HuggingFaceLLM |
| **Motore di chat** | chat/chat_engine.py — ChatEngine |
| **Contesto manuale** | knowledge_base_delta.md caricata automaticamente per domande operative su manuale, Excel, Telegram, NASA/SAR, API e troubleshooting |
| **Memoria conversazione** | Per-utente in dizionario in-memory (reset con /reset o  Reset) |
| **System prompt** | Esperto agronomico italiano: diagnosi piante, sensori DELTA, consigli |
| **Token HuggingFace** | DELTA_HF_TOKEN nel file .env — verificato all'avvio |
| **Validazione token** | HfApi.whoami() — log: 'Token HF valido — utente: <username>' |

Il motore LLM puo rispondere anche a domande operative sul Manuale DELTA Plant senza che l'utente debba scrivere esplicitamente la parola 'manuale'. Esempi tipici: export Excel, comandi Telegram, NASA-ISRO/SAR, preflight, API, troubleshooting e indice del manuale. Quando la risposta richiede una formula o una definizione presenti nella knowledge base, DELTA restituisce il testo tecnico coerente con il manuale corrente.

## 14b.2 Configurazione token HuggingFace

Per usare il modello Qwen/Qwen3-14B è necessario un token HuggingFace con permesso 'Make calls to Inference Providers'. Il token va inserito nel file .env:

> .env — TOKEN HF

```
# Nel file .env nella directory del progetto
HF_API_TOKEN=hf_xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
HF_MODEL_NAME=Qwen/Qwen3-14B

# Il sistema valida il token all'avvio:
# [INFO] delta.huggingface_llm: Token HF valido — utente: TuoNomeHF

# Se il token non è valido o assente, ChatEngine segnala backend non disponibile.
```

Modello HuggingFace configurato (senza rotazione automatica):

- Qwen/Qwen3-14B (default consigliato)
- Modificabile via variabile ambiente HF_MODEL_NAME
- Thinking mode Qwen disabilitato via API per mantenere formato risposta compatibile con DELTA

**ACCESSO AL MODELLO QWEN**

Qwen/Qwen3-14B usa la distribuzione HuggingFace pubblica del modello Qwen3 e non dipende dalla licenza gated Meta di Llama. E comunque necessario un token con permesso 'Make calls to Inference Providers' e un provider HF disponibile per il modello configurato. Pagina modello: https://huggingface.co/Qwen/Qwen3-14B

Copyright © 2026 Paolo Ciccolella. All rights reserved.

### 14b.3 Pulsante [BLU] Chiedi a DELTA — Telegram

Il pulsante [BLU] Chiedi a DELTA e il primo pulsante del menu principale Telegram (riga intera, posizione prominente). Avvia una sessione di chat interattiva con il modello LLM nel contesto del bot @DELTAPLANO_bot.

| | |
|---|---|
| **Entry point** | Callback CMD_CHAT -> ConversationHandler STATE_CHAT_WAITING |
| **Comando alternativo** | /chat -- attiva la stessa sessione di chat |
| **Stato ConversationHandler** | STATE_CHAT_WAITING = 21 (nuovo, fuori dal range standard) |
| **Priorita handler** | Registrato PRIMA dei callback globali -- non intercettato dal menu |
| **Risposta visiva** | Typing action (bot 'sta scrivendo') durante elaborazione LLM |
| **Pulsanti in chat** | [ROSSO] Termina chat \| [RESET] Reset (azzerano la conversazione) |
| **Uscita** | [ROSSO] Termina chat, /chiudi, o callback CHAT_EXIT |
| **Persistenza memoria** | Conversazione mantenuta per sessione utente (fino a reset/exit) |

**> FLUSSO CHAT TELEGRAM**

```
# Flusso chat Telegram
Utente: [preme il pulsante Chiedi a DELTA]
Bot:    Ciao! Sono DELTA... [mostra stato HF LLM]
Utente: Come si cura l'oidio del pomodoro?
Bot:    [typing...] L'oidio del pomodoro e causato da Erysiphe spp...
Utente: [preme Reset]
Bot:    Conversazione azzerata. Posso aiutarti con qualcos'altro?
Utente: [preme Termina chat]
Bot:    Chat terminata. Torna al menu con /menu
```

### 14b.4 Chat libera CLI (opzione [C])

Dalla CLI del sistema (menu principale, opzione [C]) è disponibile la stessa modalità chat direttamente dal terminale, utile per debug e uso senza Telegram.

- Selezionare [C] dal menu CLI per avviare la chat
- Digitare qualsiasi domanda in italiano o inglese
- Il sistema usa il backend HF configurato
- Digitare '/exit' per tornare al menu principale

### 14b.5 Preflight AI — verifica LLM

Il preflight di sistema (python main.py --preflight-only) verifica anche lo stato dell'LLM cloud:

- Token HF presente in .env (OK/KO)
- Token HF valido (HfApi.whoami()) (OK/KO)
- Modello HF raggiungibile (ping InferenceClient) (OK/KO)
- Se KO: ChatEngine segnala backend non disponibile

**FUNZIONAMENTO OFFLINE**

Se il Raspberry Pi non ha accesso a internet, il backend HuggingFace non risponde. Nel ramo orchestrator, se Ollama e attivo localmente, puo essere usato come backend secondario. Per abilitare l'LLM cloud assicurarsi che il Pi abbia accesso HTTPS a router.huggingface.co.

Copyright © 2026 Paolo Ciccolella. All rights reserved.

# 15. SICUREZZA E PANNELLO AMMINISTRATORE

DELTA implementa un sistema di autenticazione a doppio livello: l'operatore accede all'intero software tramite il file 'AVVIO DELTA' senza necessita di credenziali, mentre le funzioni amministrative sensibili sono protette da password crittografata.

## 15.1 File di Avvio — AVVIO DELTA

Il file 'AVVIO DELTA.command' (macOS) o 'AVVIO_DELTA.py' (tutti i sistemi) e il punto di ingresso ufficiale per l'operatore. E sufficiente un doppio clic su 'AVVIO DELTA.command' per avviare l'intero sistema, incluso l'accesso hardware (sensori, camera, NPU). Non e richiesta alcuna password per avviare il software.

| | |
|---|---|
| **AVVIO DELTA.command** | Script macOS a doppio clic. Apre il terminale e avvia DELTA automaticamente. Adatto alla distribuzione agli operatori. |
| **AVVIO_DELTA.py** | Launcher Python universale (Windows, Linux, macOS). Eseguire con: python3 AVVIO_DELTA.py |

```
> AVVIO DELTA
# macOS — doppio clic su:
  AVVIO DELTA.command

# Qualsiasi sistema operativo:
  python3 AVVIO_DELTA.py
```

**[!] ATTENZIONE**

IMPORTANTE — L'operatore deve sempre avviare DELTA tramite il file 'AVVIO DELTA' e mai modificare o eseguire direttamente i file del codice sorgente. Questo garantisce l'integrita del sistema e la corretta inizializzazione di tutti i moduli hardware e software.

## 15.2 Sistema di Autenticazione

Le credenziali di accesso sono gestite dal modulo 'core/auth.py'. La password viene salvata in 'data/auth.json' con hash PBKDF2-SHA256 e salt casuale a 256 bit — lo standard industriale per la protezione di credenziali. Il file auth.json viene creato automaticamente al primo avvio con la password di default.

| | |
|---|---|
| **Algoritmo** | PBKDF2-SHA256 con 260.000 iterazioni |
| **Salt** | 256 bit (32 byte) casuale per installazione |
| **Storage** | data/auth.json (hash + salt, mai in chiaro) |
| **Confronto** | hmac.compare_digest — resistente a timing attack |
| **Password default** | Impostata al primo avvio del sistema |

## 15.3 Pannello Amministratore

Il Pannello Amministratore e accessibile dal menu principale selezionando l'opzione [7]. Richiede la password di amministratore. Da questo pannello e possibile eseguire operazioni avanzate di gestione.

```
> PANNELLO AMMINISTRATORE
═══ PANNELLO AMMINISTRATORE ═══
  [1] Cambia password
  [2] Visualizza log
  [3] Statistiche database
  [4] Configurazione sistema
```

Copyright © 2026 Paolo Ciccolella. All rights reserved.

```
[5] Backup database
[6] Reset progressi Academy
[7] Pubblica su GitHub     <- README, RELEASE, tag, push
[8] Scientists Telegram (autorizzazioni)
[9] Programmazione orario avvio/uscita (cron)
[10] Statistiche inferenza DELTAPLANO_bot
[11] Monitor visitatori deltaplant.ai
[0] Esci dal pannello
```

| | |
|---|---|
| **[1] Cambia password** | Modifica la password amministratore. Richiede la password corrente e la nuova (minimo 8 caratteri). La nuova password viene salvata immediatamente con un nuovo salt casuale. |
| **[2] Visualizza log** | Mostra le ultime 50 righe del file di log piu recente nella cartella logs/. Utile per diagnosticare errori di sistema. |
| **[3] Statistiche database** | Visualizza il numero di record per ogni tabella del database SQLite (delta.db): diagnosi, campioni fine-tuning, log, ecc. |
| **[4] Configurazione sistema** | Visualizza i parametri di configurazione attivi: modello AI, sensori, visione/camera, quantum oracle, API REST e Telegram. |
| **[5] Backup database** | Crea una copia del database delta.db nella cartella exports/ con timestamp. Formato: delta_backup_YYYYMMDD_HHMMSS.db |
| **[6] Reset Academy** | Azzera tutti i progressi, punteggi e badge della DELTA Academy per l'operatore corrente. Operazione irreversibile. |
| **[7] Pubblica su GitHub** | Avvia il wizard di pubblicazione automatica su GitHub. Raccoglie metadati dal software, genera README.md e RELEASE.md aggiornati, crea un tag git con versione incrementale e fa push su origin/main. Vedi sezione 15.6 per i dettagli. |
| **[8] Scientists Telegram** | Gestisce i nickname Telegram autorizzati a usare il bot (lista in data/telegram_scientists.local.json, file locale non tracciato).<br><br>Novità 2026:<br>- Log automatico degli accessi negati (file logs/telegram_denied.log).<br>- Dal pannello admin puoi vedere gli accessi negati e aggiungere rapidamente utenti bloccati alla whitelist.<br>- La normalizzazione avanzata dei nickname permette di tollerare piccole variazioni (underscore, punti, trattini, spazi e maiuscole vengono ignorati). |
| **[9] Programmazione orario avvio/uscita** | Permette di programmare l'orario automatico di avvio e di uscita di DELTA sulla shell del Raspberry Pi 5 tramite crontab. E possibile impostare, modificare e rimuovere separatamente l'orario di avvio (esegue main.py) e l'orario di uscita (pkill). Gli orari si inseriscono in formato 24h (HH:MM). Le entry vengono salvate nel crontab dell'utente con tag identificativi DELTA_SCHEDULE_START e DELTA_SCHEDULE_STOP, senza interferire con altri job cron esistenti. |
| **[10] Statistiche inferenza DELTAPLANO_b** | Produce un report per utente autorizzato sul traffico LLM del bot: richieste, errori, modelli usati, ultimo accesso e istogramma ASCII. Legge anche log |

Copyright © 2026 Paolo Ciccolella. All rights reserved.

| | |
|---|---|
| | ruotati, deduplica duplicati e salva un report testuale in logs/deltaplano_inference_stats.txt. |
| **[11] Monitor visitatori deltaplant.ai** | Apre il monitor analytics first-party del sito pubblico. Mostra visitatori unici, sessioni, page view, CTA, referrer, lingue, dispositivi ed eventi recenti a 24h/7g/30g, leggendo prima il report live protetto dell'API pubblica e usando il fallback SQLite locale solo se necessario. Include export JSON e diagnostica del report remoto. I dati sono raccolti solo dopo consenso analytics; sul sito pubblico il consenso puo essere dato dal banner Privacy & cookies o, su mobile/viewport compatti, dal pulsante Privacy. |

## 15.4 Cambio Password

Per cambiare la password amministratore accedere al Pannello Amministratore e selezionare l'opzione [1]. La nuova password deve avere minimo 8 caratteri. In caso di smarrimento della password, e possibile resettarla eliminando il file data/auth.json: al successivo avvio verra ricreato con la password di default.

**[!] ATTENZIONE**

SICUREZZA — Conservare la password amministratore in un luogo sicuro. Non condividerla con gli operatori. Si raccomanda di cambiare la password di default al primo utilizzo del sistema in ambiente di produzione.

## 15.5 Accesso Hardware

Il launcher 'AVVIO DELTA' ha accesso completo a tutti i componenti hardware del sistema senza necessita di autenticazione. Questo include:

- Camera Module — acquisizione immagini foglia/fiore/frutto
- BME680 — temperatura, umidita, pressione, VOC (I2C)
- VEML7700 — illuminanza lux (I2C)
- SCD41 — CO2 ppm (I2C)
- ADS1115 — pH e conducibilita elettrica via ADC (I2C)
- AI HAT 2+ (Hailo-8) — inferenza NPU per analisi visiva

**HARDWARE E SICUREZZA**

L'accesso hardware avviene esclusivamente tramite il launcher ufficiale. I driver I2C, la camera e il NPU sono inizializzati automaticamente all'avvio. In caso di hardware mancante, il sistema opera in modalita simulazione con dati sintetici, senza bloccare il software.

Copyright © 2026 Paolo Ciccolella. All rights reserved.

# 16. INPUT IMMAGINI DA CARTELLA — MODALITÀ NO-CAMERA

DELTA supporta l'analisi di immagini fornite manualmente dall'utente tramite una cartella di input dedicata. Questa modalità è particolarmente utile quando la videocamera non è fisicamente disponibile, è in manutenzione, oppure quando si desidera analizzare immagini acquisite in precedenza o con dispositivi esterni (smartphone, fotocamera DSLR, microscopi digitali).

## 16.1 Cartella di input

| | |
|---|---|
| **Percorso cartella** | /home/proctor81/Desktop/DELTA-PLANT/input_images |
| **Formati supportati** | .jpg, .jpeg, .png, .bmp, .tiff, .webp |
| **Selezione immagine** | Interattiva — lista numerata o analisi di tutte |
| **Risoluzione minima** | Qualsiasi — DELTA ridimensiona automaticamente a 224x224 |
| **Salvataggio frame** | Le immagini caricate NON vengono copiate nella cartella captures |

## 16.2 Come inserire immagini

Per caricare immagini da analizzare nella cartella di input:

- Aprire la cartella: /home/proctor81/Desktop/DELTA-PLANT/input_images
- Copiare le immagini da analizzare (JPG, PNG, BMP, TIFF o WEBP)
- Avviare DELTA e selezionare [1] Avvia diagnosi pianta
- Rispondere 'S' alla domanda 'Caricare immagine da cartella di input?'
- Selezionare l'immagine dalla lista numerata oppure premere 'A' per analizzarle tutte

> TERMINALE

```
# Copia immagini in input_images/ dalla riga di comando
cp /percorso/immagine.jpg ~/DELTA/input_images/

# Da dispositivo USB montato
cp /media/pi/USB/scansioni/*.jpg ~/DELTA/input_images/

# Verifica contenuto cartella
ls -lh ~/DELTA/input_images/
```

## 16.3 Flusso di selezione nel menu CLI

Durante il flusso di diagnosi [1], il sistema presenta le seguenti opzioni:

| | |
|---|---|
| **Camera disponibile** | Il sistema chiede se usare la camera o la cartella di input. Default: camera. Rispondere 'S' per usare la cartella. |
| **Camera NON disponibile** | Il sistema passa automaticamente alla modalita cartella di input. Viene mostrata la lista delle immagini disponibili. |
| **Lista immagini** | Ogni immagine viene mostrata con nome e dimensione in KB. Inserire il numero per selezionare, oppure 'A' per tutte. |
| **Modalita sequenziale (A)** | Analizza tutte le immagini in cartella in sequenza. Ogni risultato viene salvato nel database. Riepilogo finale mostrato e ritorno diretto al menu principale (nessun annullamento). |

Copyright © 2026 Paolo Ciccolella. All rights reserved.

## 16.4 Voce [8] — Gestione cartella dal menu principale

Il menu principale include la voce [8] per visualizzare rapidamente il contenuto della cartella di input senza avviare una diagnosi:

- Mostra il percorso assoluto della cartella
- Elenca tutte le immagini con nome e dimensione
- Informa se la cartella e vuota con istruzioni su come procedere
- Ricorda i formati supportati e come avviare la diagnosi

## 16.5 Integrazione con dispositivi esterni

La modalita cartella di input si integra naturalmente con la catena di acquisizione immagini esistente in azienda o laboratorio:

- Fotocamere DSLR/mirrorless collegate tramite cavo USB o scheda SD
- Smartphone via connessione WiFi (app di trasferimento file)
- Scanner o microscopi digitali con output su file
- Sistemi di telerilevamento con immagini NIR/RGB
- Database di immagini storiche per analisi retrospettiva
- Pipeline automatizzate con script cron per analisi notturna

**ANALISI BATCH AUTOMATICA**

Per analizzare automaticamente tutte le immagini in una cartella senza interazione manuale, e possibile usare setup_raspberry.py o uno script cron. Tutti i risultati vengono salvati nel database SQLite e nel file Excel.

Copyright © 2026 Paolo Ciccolella. All rights reserved.

# 17. INSTALLAZIONE AUTOMATICA SU RASPBERRY PI 5

DELTA include uno script di installazione automatica (install_raspberry.sh) e uno script di configurazione post-installazione (setup_raspberry.py) che automatizzano l'intero processo di deployment su Raspberry Pi 5 con Raspberry Pi OS (64-bit, Bookworm o superiore).

## 17.1 Prerequisiti hardware

- Raspberry Pi 5 (4 GB, 8 GB o 16 GB RAM)
- MicroSD o SSD esterna da 256 GB con Raspberry Pi OS (64-bit) installato
- AI HAT 2+ (opzionale — per inferenza accelerata via NPU)
- Pi Camera Module 3 o webcam USB (opzionale — alternativamente: cartella input)
- Sensori I2C Adafruit (opzionale — alternativamente: modalita simulata)
- Connessione Internet per il download dei pacchetti
- Alimentatore originale Raspberry Pi USB-C 5V/5A (minimo 27W)

## 17.2 Procedura di installazione rapida

Eseguire i seguenti comandi da terminale sulla Raspberry Pi. Lo script richiede i privilegi di root (sudo) per installare i pacchetti e configurare i servizi di sistema.

> INSTALLAZIONE

```
# 1. Copia i sorgenti DELTA sulla Raspberry Pi
scp -r DELTA/ pi@raspberrypi.local:~/DELTA

# 2. Connettiti alla Raspberry Pi
ssh pi@raspberrypi.local

# 3. Entra nella directory DELTA
cd ~/DELTA

# 4. Rendi eseguibile lo script
chmod +x install_raspberry.sh

# 5. Avvia l'installazione automatica
sudo ./install_raspberry.sh
```

## 17.3 Cosa fa install_raspberry.sh (9 step)

| | |
|---|---|
| **Step 1 — Aggiornamento OS** | apt-get update + upgrade. Sistema sempre aggiornato all'ultima versione. |
| **Step 2 — Dipendenze sistema** | Installa: Python 3.11, OpenCV, libcamera, i2c-tools, picamera2, font. |
| **Step 3 — Configurazione hardware** | Abilita I2C, SPI, Camera in /boot/firmware/config.txt. Aggiunge overlay AI HAT 2+. Aggiunge utente ai gruppi hardware. |
| **Step 4 — Copia sorgenti** | Copia i sorgenti DELTA nella directory target. Crea tutte le sottodirectory necessarie (input_images, models, exports...). |
| **Step 5 — Ambiente virtuale** | Crea .venv con --system-site-packages per accedere a picamera2 di sistema. |
| **Step 6 — Dipendenze Python** | pip install -r requirements.txt (include ai-edge-litert==1.2.0) + librerie Adafruit. |
| **Step 7 — Generazione manuale** | Genera automaticamente il PDF del Manuale Utente aggiornato. |

Copyright © 2026 Paolo Ciccolella. All rights reserved.

| | |
|---|---|
| **Step 6b — File .env** | Crea automaticamente .env da .env.example se assente. Configurare DELTA_TELEGRAM_TOKEN per il bot @DELTAPLANO_bot. |
| **Step 8 — Servizio systemd** | Crea e abilita il servizio delta.service (con network-online.target) per avvio automatico al boot. Alternativa rapida: install_service.sh. |
| **Step 9 — Comando rapido** | Installa il comando 'delta' in /usr/local/bin per avvio rapido da terminale. |

## 17.4 Configurazione post-installazione (setup_raspberry.py)

Dopo l'installazione, eseguire lo script di verifica per controllare che tutti i componenti siano correttamente installati:

```
> CONFIGURAZIONE
# Eseguire dopo install_raspberry.sh
cd ~/DELTA
source .venv/bin/activate
python setup_raspberry.py
```

Lo script verifica e configura:

- Versione Python (>= 3.11 richiesta)
- Dipendenze Python obbligatorie e opzionali
- Disponibilita camera (picamera2 / OpenCV / modalita cartella)
- Scansione bus I2C per rilevare sensori collegati
- Struttura directory (input_images, models, exports, logs...)
- Generazione del Manuale PDF aggiornato

## 17.5 Installazione autostart (install_service.sh)

Lo script install_service.sh e la procedura consigliata per configurare l'avvio automatico di DELTA (e del bot @DELTAPLANO_bot) ad ogni accensione del Raspberry Pi, senza dover reinstallare l'intero sistema.

```
> AUTOSTART
# Dalla directory del progetto
cd ~/DELTA

# Installazione servizio (richiede sudo)
sudo bash install_service.sh

# Rimozione autostart
sudo bash install_service.sh --remove
```

Lo script esegue automaticamente: verifica del file .env e del token Telegram, abilitazione di network-online.target (rete disponibile prima del bot), scrittura del file /etc/systemd/system/delta.service con i percorsi reali, sostituzione dell'interprete effettivo del venv nel comando ExecStart, enable + start del servizio.

Nel workspace VS Code il task predefinito 'DELTA: Preflight + Avvia Sistema' usa lo stesso comando validato del servizio: preflight iniziale, API attiva, Telegram attivo e runtime in daemon. Questo evita avvii parziali in cui il modello parte ma il bot Telegram resta disabilitato.

## 17.6 Configurazione token Telegram

Il bot @DELTAPLANO_bot richiede un token valido da @BotFather. Il token va inserito nel file .env nella directory di progetto:

Copyright © 2026 Paolo Ciccolella. All rights reserved.

> TOKEN TELEGRAM

```
# Crea il file .env dal template
cp .env.example .env

# Modifica con il tuo editor preferito
nano .env
# Riga da impostare:
DELTA_TELEGRAM_TOKEN=<token_da_BotFather>

# Riavvia il servizio per applicare
sudo systemctl restart delta
```

## 17.7 Gestione del servizio systemd

> GESTIONE SERVIZIO

```
# Avvia DELTA come servizio
sudo systemctl start delta

# Ferma il servizio
sudo systemctl stop delta

# Riavvio (es. dopo modifica .env)
sudo systemctl restart delta

# Stato del servizio
sudo systemctl status delta

# Log in tempo reale
journalctl -u delta -f

# Avvio manuale interattivo (senza servizio)
delta
```

## 17.6 Avvio su Raspberry Pi OS senza camera

Se la camera non e disponibile o non e ancora collegata, DELTA si avvia normalmente e attiva automaticamente la modalita cartella di input immagini. L'operatore puo:

- Copiare immagini in ~/DELTA/input_images/
- Avviare una diagnosi dal menu [1]
- Selezionare un'immagine dalla lista o analizzarle tutte in batch
- Collegare la camera in un secondo momento: il sistema la rileva automaticamente

**RIAVVIO DOPO INSTALLAZIONE**

Al termine di install_raspberry.sh viene richiesto di riavviare il sistema per attivare le modifiche all'hardware (I2C, SPI, Camera, AI HAT overlay).

sudo reboot

Dopo il riavvio DELTA si avvia automaticamente come servizio di sistema. Per accedere all'interfaccia CLI: aprire un terminale e digitare 'delta'.

**[!] ATTENZIONE**

Copyright © 2026 Paolo Ciccolella. All rights reserved.

COMPATIBILITA: install_raspberry.sh richiede Raspberry Pi OS (64-bit) versione Bookworm (Debian 12) o superiore. Non e compatibile con versioni precedenti a 32-bit o con distribuzioni Linux alternative. Verificare la versione con: cat /etc/os-release

Copyright © 2026 Paolo Ciccolella. All rights reserved.

# 18. MLOPS — ADDESTRAMENTO E MIGLIORAMENTO CONTINUO

Il ciclo di miglioramento continuo del modello AI di DELTA segue il paradigma MLOps (Machine Learning Operations): raccolta dati, addestramento, conversione, validazione e deploy. Questo capitolo descrive le procedure operative standard (SOP) per l'operatore responsabile del miglioramento del modello.

## 18.1 Ciclo di Miglioramento Continuo

| | |
|---|---|
| **Fase 1 — Raccolta dataset** | Acquisire immagini di piante con etichetta di classe (nome malattia o 'Sano'). Organizzare in cartelle: datasets/training/<NomeClasse>/immagine.jpg. Minimo 50–100 immagini per classe, preferibilmente 200+. |
| **Fase 2 — Addestramento Keras** | Eseguire ai/train_keras_classifier.py per addestrare un modello MobileNetV2 con transfer learning. Il modello viene salvato in models/plant_disease_model.keras. |
| **Fase 3 — Conversione TFLite** | Eseguire ai/convert_keras_to_tflite.py per convertire il modello Keras in formato TFLite (float16 consigliato per il modello principale, INT8 opzionale per scenari specifici NPU). |
| **Fase 4 — Preflight Validazione** | Eseguire python main.py --preflight-only per verificare che il modello superi la soglia minima di confidenza su un'immagine di test. Il sistema blocca il deploy se la soglia non e soddisfatta. |
| **Fase 5 — Deploy** | Copiare plant_disease_model_39classes.tflite in models/ sul dispositivo target. Riavviare DELTA. Il nuovo modello viene caricato automaticamente all'avvio. |

**PREREQUISITI**

Per eseguire addestramento e conversione e necessario:

- tensorflow >= 2.13 installato nell'ambiente virtuale (.venv)
- Dataset organizzato in cartelle per classe in datasets/training/
- Python .venv attivato: source .venv/bin/activate
- Almeno 4 GB RAM disponibili (8+ GB raccomandati per training)

## 18.2 Struttura del Dataset

Il dataset di training deve essere organizzato secondo la struttura folder-per-class standard di Keras: ogni sottocartella rappresenta una classe e contiene le immagini di quella classe.

> STRUTTURA DATASET

```
datasets/training/
├── Sano/
│   ├── sano_001.jpg
│   ├── sano_002.jpg
│   └── ...  (min. 50 immagini)
├── Oidio/
│   ├── oidio_001.jpg
│   └── ...
├── Peronospora/
│   └── ...
└── Muffa_grigia/
```

Copyright © 2026 Paolo Ciccolella. All rights reserved.

```
└── ...
```

| | |
|---|---|
| **Formato immagini** | JPG, PNG, BMP, TIFF — qualsiasi risoluzione |
| **Immagini per classe** | Minimo: 50 \| Raccomandato: 200+ \| Ottimale: 500+ |
| **Classi minime** | 2 (binario: Sano / Malato) — nessun limite massimo |
| **Bilanciamento** | Classi bilanciate preferibili; squilibri fino a 1:3 accettabili |
| **Qualita** | Immagini nitide, ben illuminate, organo centrato nel frame |
| **Augmentation** | Automatica durante training (flip, zoom, rotazione, contrasto) |

## 18.3 Addestramento del Modello — train_keras_classifier.py

Lo script ai/train_keras_classifier.py addestra un classificatore MobileNetV2 con transfer learning in due fasi: prima solo il layer di classificazione (head), poi fine-tuning degli ultimi layer della base MobileNetV2.

> TRAINING

```
# Attivare l'ambiente virtuale
source .venv/bin/activate

# Addestramento base (consigliato per iniziare)
python ai/train_keras_classifier.py \
  --dataset datasets/training \
  --output models

# Addestramento avanzato con piu epoche e fine-tuning
python ai/train_keras_classifier.py \
  --dataset datasets/training \
  --output models \
  --epochs 30 \
  --fine-tune-epochs 20 \
  --fine-tune-layers 50 \
  --batch-size 32
```

| | |
|---|---|
| **--dataset** | Percorso cartella dataset folder-per-class |
| **--output** | Cartella output per modello e labels (default: models/) |
| **--epochs** | Epoche fase 1 — solo head (default: 20) |
| **--fine-tune-epochs** | Epoche fase 2 — fine-tuning base (default: 10) |
| **--fine-tune-layers** | Numero layer base da sbloccare in fine-tuning (default: 30) |
| **--batch-size** | Dimensione batch (default: 16, aumentare se RAM sufficiente) |
| **--img-size** | Dimensione input immagini in pixel (default: 224) |
| **--val-split** | Percentuale dati usata per validazione (default: 0.2) |

Output generati al termine dell'addestramento:

- models/plant_disease_model.keras — modello Keras completo
- models/labels_33classes_correct.txt — lista classi (una per riga, in ordine indice)
- models/training_metadata.json — metriche training, epoche, accuracy, data

## 18.4 Conversione TFLite (float16 / int8) — convert_keras_to_tflite.py

Lo script ai/convert_keras_to_tflite.py converte il modello Keras in formato TFLite ottimizzato per edge deployment. In

Copyright © 2026 Paolo Ciccolella. All rights reserved.

DELTA v3.2 il modello generale continua a usare float16 come profilo stabile di produzione, mentre il backend EfficientFormer della Pipeline X adotta anche una variante int8 pienamente validata per questo hardware.

```
> CONVERSIONE
# Conversione standard float16 (baseline produzione DELTA v3.2)
python ai/convert_keras_to_tflite.py \
  --keras-model models/plant_disease_model.keras \
  --output models/plant_disease_model_39classes.tflite \
  --quantization float16

# Conversione INT8 (opzionale, richiede dati rappresentativi)
python ai/convert_keras_to_tflite.py \
  --keras-model models/plant_disease_model.keras \
  --output models/plant_disease_model_39classes_int8.tflite \
  --quantization int8 \
  --representative-data datasets/training

# Conversione senza quantizzazione (FP32 — massima precisione, piu lento)
python ai/convert_keras_to_tflite.py \
  --keras-model models/plant_disease_model.keras \
  --output models/plant_disease_model_fp32.tflite \
  --quantization none
```

| | |
|---|---|
| **none** | FP32 — precisione massima, dimensione maggiore, piu lento su edge |
| **dynamic** | Quantizzazione dinamica — buon compromesso senza dataset calibrazione |
| **float16** | FP16 — baseline stabile del modello generale in DELTA v3.2 |
| **int8** | Quantizzazione intera INT8 — profilo edge pienamente supportato per pipeline dedicate |

## 18.4b Pipeline EfficientFormerV2-S1 — export_efficientformer_tflite.py

Per il backend ibrido CNN/ViT, DELTA include una pipeline completa PyTorch -> ONNX -> SavedModel -> TFLite. Lo script ai/export_efficientformer_tflite.py puo eseguire fine-tuning, export ONNX, conversione float16 e conversione int8 fully quantized nello stesso flusso.

```
> EFFICIENTFORMER EXPORT
# Stack opzionale per EfficientFormer
pip install -r requirements-efficientformer.txt

# Fine-tuning + export completo
python ai/export_efficientformer_tflite.py \
  --dataset-root datasets/training_33classes \
  --output-dir models \
  --mode all \
  --quantization both

# Solo export calibrato da checkpoint esistente
python ai/export_efficientformer_tflite.py \
  --dataset-root datasets/training_33classes \
  --mode export \
  --checkpoint models/efficientformer_v2_s1_33classes.pth \
```

Copyright © 2026 Paolo Ciccolella. All rights reserved.

```
--quantization both
```

- Output principali: efficientformer_v2_s1_33classes.pth, .onnx, SavedModel, TFLite float16, TFLite int8 e file labels.
- Preprocessing training/export allineato al runtime edge: mean/std = 0.5, equivalente a normalizzazione [-1, 1].
- Per INT8 serve un representative dataset reale; lo script usa di default datasets/training.
- LayerCAM richiede il checkpoint PyTorch oltre al file TFLite di inferenza e salva overlay/heatmap in exports/explanations quando la spiegabilita e attiva.

## 18.4c Benchmark ed evaluation backend multipli

```
> BENCHMARK + EVAL
# Resume end-to-end della Pipeline X
python tools/pipeline_x.py --resume

# Benchmark latency su device target
python tools/benchmark_vision_models.py \
  --model-keys generale efficientformer \
  --image models/validation_sample.jpg \
  --runs 100 --warmup 10

# Accuracy, macro-F1 e confusion matrix su validation set
python ai/evaluate_vision_backends.py \
  --dataset-root datasets/training_33classes/validation \
  --model-keys generale efficientformer \
  --output-dir logs/vision_eval
```

Questi script producono report JSON/CSV con accuracy top-1, accuracy top-3, macro-F1, classification report e confusion matrix. I numeri EfficientFormer vanno pubblicati solo dopo esecuzione sul Raspberry Pi 5 reale e sul validation set finale. Se nelle superfici documentali si usa una proiezione target EfficientFormer, va etichettata esplicitamente come non misurata e derivata dal profilo Generale con un incremento documentale contenuto (ad esempio +4%) e con un cap massimo al 100%.

## 18.5 Validazione Preflight — Gate di Deploy

Prima di distribuire un nuovo modello in produzione, il sistema esegue una validazione automatica end-to-end su un'immagine di test. Il deploy e bloccato se la confidenza e inferiore alla soglia configurata.

```
> PREFLIGHT
# Esegui solo la validazione (non avvia il sistema)
python main.py --preflight-only

# Validazione con immagine personalizzata
python main.py --preflight-only \
  --validation-image input_images/foglia_test.jpg

# Validazione con soglia minima personalizzata
python main.py --preflight-only \
  --preflight-min-confidence 0.75

# Avvia il sistema con validazione preliminare (non bloccante per il menu)
python main.py --preflight
```

Copyright © 2026 Paolo Ciccolella. All rights reserved.

| | |
|---|---|
| **Esito OK** | Confidenza >= soglia minima. Il sistema mostra classe predetta, confidenza e shape I/O. Deployment autorizzato. |
| **Esito FAIL** | Confidenza < soglia minima (PreflightGateError). Il sistema mostra un errore critico e termina con codice di uscita 1. Deployment bloccato. |
| **Soglia default** | 0.50 (50%) — configurabile in MODEL_CONFIG['preflight_min_confidence'] in core/config.py o con flag --preflight-min-confidence |

**[!] ATTENZIONE**

GATE DI DEPLOY — Se il preflight fallisce, il modello non e pronto per la produzione. Verificare la qualita del dataset, aumentare il numero di immagini per classe, controllare i log di training per segni di overfitting, e ripetere il ciclo training → conversione → preflight.

## 18.6 Riepilogo Comandi MLOps

**> PIPELINE COMPLETA**

```
# 1. Prepara il dataset
mkdir -p datasets/training_33classes/Apple_Apple_scab datasets/training_33classes/Tomato_healthy
# ... copia immagini nelle cartelle ...

# 2. Esegui la pipeline resumable end-to-end
python tools/pipeline_x.py --resume

# 3. In alternativa, esporta EfficientFormer da checkpoint esistente
python ai/export_efficientformer_tflite.py \
  --dataset-root datasets/training_33classes --output-dir models --mode export --quantization both \
  --checkpoint models/efficientformer_v2_s1_33classes.pth

# 4. Valida il modello (gate di deploy)
python main.py --preflight-only

# 4b. Benchmark, evaluation, dissemination e manuale
python tools/benchmark_vision_models.py --model-keys generale efficientformer --image models/validation_sample.jpg
python ai/evaluate_vision_backends.py --dataset-root datasets/training_33classes/validation --model-keys generale efficientformer
python Manuale/genera_manuale.py

# 5. Avvia DELTA con il nuovo modello
python main.py --preflight
```

**LAB MLOPS IN DELTA ACADEMY**

La DELTA Academy (menu [6] → [7]) include il Lab MLOps Operatore interattivo: un percorso formativo guidato con checklist operativa, mini-quiz e comandi pratici per apprendere il ciclo di miglioramento continuo senza eseguire training reale. Completare il Lab MLOps prima di eseguire la pipeline in produzione.

Copyright © 2026 Paolo Ciccolella. All rights reserved.

# 19. PIPELINE ALTERNATIVA LEGACY (7 CLASSI) — DOWNLOAD AUTOMATICO

In assenza di un dataset proprietario, DELTA include lo script ai/download_pretrained_model.py che scarica automaticamente il dataset PlantVillage tramite TensorFlow Datasets, addestra un modello MobileNetV2 con transfer learning sulle classi DELTA, e converte il risultato in TFLite INT8 pronto per il deploy. Non e richiesto un account Kaggle. Questa pipeline e separata dal modello principale v3.2 leaf-only a 33 classi.

## 19.1 Prerequisiti

- Python 3.12 installato (TensorFlow 2.x non supporta Python 3.13+)
- TensorFlow >= 2.13 e tensorflow-datasets installati nell'ambiente .venv
- protobuf >= 6.x presente (workaround incluso nello script — vedi 19.2)
- Connessione Internet (~820 MB per il download del dataset, solo prima volta)
- Almeno 4 GB di RAM libera + ~2 GB di spazio disco
- Tempo stimato: 40-80 minuti su CPU Apple Silicon / Intel (8 epoche)

> INSTALLAZIONE DIPENDENZE

```
# L'ambiente .venv gia include Python 3.12 e TensorFlow — nessun venv aggiuntivo
cd ~/DELTA

# Installa tensorflow-datasets (se non presente)
.venv/bin/pip install tensorflow-datasets

# Verifica
.venv/bin/python -c "import tensorflow as tf; print(tf.__version__)"
.venv/bin/python -c "import tensorflow_datasets as tfds; print(tfds.__version__)"
```

## 19.2 Avvio del download e training

Lo script esegue le seguenti operazioni in sequenza:
1) Scarica PlantVillage (~820 MB, solo alla prima esecuzione — poi in cache)
2) Mappa le 38 classi PlantVillage sulle 7 classi DELTA
3) Addestra MobileNetV2 in 2 fasi (head + fine-tuning)
4) Salva il modello Keras in models/plant_disease_model.keras
5) Converte e quantizza in TFLite INT8: models/plant_disease_model.tflite
6) Genera labels.txt con le 7 classi DELTA

NOTA: protobuf >= 5 (installato con TensorFlow 2.20+) introduce un'incompatibilita con tensorflow-datasets 4.x. La variabile d'ambiente PROTOCOL_BUFFERS_PYTHON_IMPLEMENTATION=python e gia impostata dallo script in automatico; non e necessaria alcuna azione manuale.

> AVVIO DOWNLOAD + TRAINING

```
cd ~/DELTA

# Avvio con parametri di default (8 epoche, batch 32)
.venv/bin/python ai/download_pretrained_model.py --epochs 8 --output models

# Oppure in background con log su file
.venv/bin/python ai/download_pretrained_model.py \
  --epochs 8 --output models --log-level INFO \
```

Copyright © 2026 Paolo Ciccolella. All rights reserved.

```
  > logs/download_pretrained.log 2>&1 &

# Monitora il progresso
tail -f logs/download_pretrained.log
```

## 19.3 Parametri disponibili

| | |
|---|---|
| **--output** | Directory output (default: models/) |
| **--epochs** | Epoche totali: prime 3 = solo head, resto = fine-tuning (default: 8) |
| **--batch-size** | Batch size per il training (default: 32) |
| **--img-size** | Dimensione input immagini (default: 224 px) |
| **--fine-tune-layers** | Quanti layer finali del backbone sbloccare (default: 30) |
| **--data-dir** | Cache tensorflow_datasets (default: ~/tensorflow_datasets) |
| **--no-quantize** | Salta la conversione TFLite — salva solo il .keras |
| **--log-level** | Verbosita: DEBUG / INFO / WARNING / ERROR (default: INFO) |

## 19.4 Mappatura classi PlantVillage → DELTA

PlantVillage contiene 38 classi su 14 colture. Lo script le raggruppa automaticamente nelle 7 classi DELTA. Le classi senza equivalente (es. spider mites, Huanglongbing) vengono scartate.

| | |
|---|---|
| **Sano** | Tutte le classi 'healthy' di ogni coltura |
| **Peronospora** | late_blight, leaf_blight, esca |
| **Oidio** | powdery_mildew |
| **Muffa_grigia** | leaf_mold |
| **Alternaria** | early_blight, cercospora, northern_leaf_blight, apple_scab, target_spot, septoria, leaf_scorch, bacterial_spot, black_rot |
| **Ruggine** | rust (tutte le varianti) |
| **Mosaikovirus** | mosaic, yellow_leaf_curl_virus |

## 19.5 Output generato

Al termine dello script, la cartella models/ conterrà:

- plant_disease_model.keras — modello Keras completo (~21 MB, per ulteriore fine-tuning)
- plant_disease_model.tflite — modello TFLite INT8 (~2.6 MB, per deploy su Raspberry Pi / Hailo NPU)
- labels.txt — 7 classi DELTA, una per riga, nell'ordine corretto per l'inferenza

Accuratezza attesa su validation set con parametri di default (8 epoche, MobileNetV2):
Fase 1 — head only (3 epoche): val_accuracy ~93-95%
Fase 2 — fine-tuning (5 epoche): val_accuracy ~97-98%

**AVVIO DELTA DOPO IL DOWNLOAD**

Una volta completato lo script, DELTA rileva automaticamente il modello alla prossima avvio e passa dalla modalita degradata a quella operativa completa. Verificare con il preflight:

```
.venv/bin/python main.py --preflight-only
```

Se il preflight passa, il modello e pronto per la produzione.

**[!] ATTENZIONE**

Copyright © 2026 Paolo Ciccolella. All rights reserved.

NOTA SULLA VERSIONE PYTHON — TensorFlow 2.x richiede Python <= 3.12. L'ambiente .venv del progetto usa Python 3.12 ed e gia compatibile. Non e necessario creare un ambiente virtuale separato per il training.

Copyright © 2026 Paolo Ciccolella. All rights reserved.

# 20. GITHUB PUBLISHER — PUBBLICAZIONE AUTOMATICA

Il modulo GitHub Publisher (interface/github_publisher.py) consente di pubblicare automaticamente DELTA Plant su GitHub con un solo click dal Pannello Amministratore. Raccoglie in autonomia i metadati tecnici dal software in esecuzione — versione, modello AI, dipendenze, changelog git — e genera un README.md e un RELEASE.md aggiornati, crea un tag git con versione incrementale e fa push su origin/main, senza richiedere alcun intervento manuale. I dati operativi locali (diagnosi, analisi, statistiche database) non vengono mai inclusi nei file pubblicati, garantendo la piena privacy delle informazioni raccolte in campo.

## 20.1 Accesso al GitHub Publisher

Il publisher e accessibile esclusivamente dal Pannello Amministratore (richiede password). Dal menu principale selezionare [7] Pannello Amministratore, quindi [7] Pubblica su GitHub.

```
> PERCORSO DI ACCESSO
MENU PRINCIPALE
  [7] Pannello Amministratore  <- autenticazione richiesta

PANNELLO AMMINISTRATORE
  [7] Pubblica su GitHub

GITHUB PUBLISHER
  [1] Pubblicazione completa  (README + RELEASE + tag + push)
  [2] Solo README             (aggiorna README.md senza nuovo tag)
  [3] Anteprima dati          (mostra dati senza modificare nulla)
  [0] Annulla
```

## 20.2 Raccolta Automatica dei Metadati

Il publisher non richiede alcuna configurazione manuale: raccoglie tutto ciò che serve direttamente dal software in esecuzione.

| | |
|---|---|
| **Git (branch/remote/tag)** | Branch corrente, URL repository, ultimo tag git, conteggio commit, changelog dei commit dall'ultimo tag fino a HEAD. |
| **Modello AI (TFLite)** | Classi diagnostiche da models/labels_33classes_correct.txt, dimensione file .tflite in KB, shape di input (es. 224x224x3), stato operativo del modello. |
| **Database SQLite (delta.db)** | Totale diagnosi archiviate, diagnosi reali (non simulate), top-5 classi piu frequenti, distribuzione livelli di rischio. |
| **requirements.txt** | Lista delle dipendenze Python attive, con versioni fissate. |
| **core/config.py** | Parametri chiave: soglia confidenza AI, thread NPU, range temperatura/umidita ottimali, qubit Oracle di Grover. |

## 20.3 File Generati Automaticamente

Dalla raccolta dati vengono prodotti due file Markdown e rigenerato il manuale PDF prima del push.

| | |
|---|---|
| **README.md** | Descrizione del progetto con badge tecnici (versione, Python, piattaforma), tabella classi diagnostiche, sezioni installazione, avvio, architettura moduli, struttura directory, requisiti hardware/software e nota privacy. Nessun dato |

Copyright © 2026 Paolo Ciccolella. All rights reserved.

| | |
|---|---|
| | operativo incluso. |
| **RELEASE.md** | Note di rilascio con version tag, data, changelog automatico estratto da git log, dimensione modello e numero classi. Nessuna statistica del database inclusa. |
| **Manuale/DELTA_Manuale_Utente.pdf** | Il manuale PDF viene rigenerato automaticamente prima del push tramite Manuale/genera_manuale.py per garantire che sia sempre allineato con il codice pubblicato. |

## 20.4 Flusso di Pubblicazione Completa ([1])

La modalita 'Pubblicazione completa' esegue in sequenza tutte le operazioni necessarie per aggiornare il repository GitHub.

```
> FLUSSO PUBBLICAZIONE COMPLETA
1. Raccolta automatica di tutti i metadati dal software
2. Generazione README.md aggiornato
3. Generazione RELEASE.md con changelog
4. Rigenerazione Manuale/DELTA_Manuale_Utente.pdf
5. Richiesta versione  (suggerita automaticamente, es. v3.2.1)
6. Conferma prima del push
7. git add README.md RELEASE.md Manuale/DELTA_Manuale_Utente.pdf
8. git commit -m "chore(release): <versione>"
9. git tag <versione>
10. git push origin main --tags
```

## 20.5 Versione Incrementale Automatica

Se esiste un tag git precedente (es. v3.2.0), il publisher suggerisce automaticamente la patch successiva (v3.2.1). L'amministratore puo accettare il suggerimento o inserire una versione personalizzata.

| | |
|---|---|
| **Nessun tag precedente** | Propone v2.0.0 come versione iniziale |
| **Tag precedente: v2.0.0** | Propone v2.0.1 (incremento patch) |
| **Tag precedente: v2.1.3** | Propone v2.1.4 (incremento patch) |
| **Formato accettato** | vX.Y.Z  (es. v2.0.1, v3.0.0) |

## 20.6 Modalita 'Solo README' ([2])

Aggiorna il README.md con i dati tecnici piu recenti (classi modello, dipendenze, configurazione) e fa push su origin/main senza creare un nuovo tag git. Utile per aggiornare la homepage del repository senza incrementare la versione del software. Anche in questa modalita nessun dato operativo locale viene incluso.

## 20.7 Anteprima Dati ([3])

Mostra a schermo tutti i dati che verrebbero pubblicati, senza scrivere alcun file ne fare push. Utile per verificare il contenuto prima della pubblicazione.

```
> ESEMPIO ANTEPRIMA
—— ANTEPRIMA DATI RACCOLTI ——
Repository:   https://github.com/Proctor81/DELTA-PLANT
Branch:       main
Ultimo tag:   v3.2.0  (o 'nessun tag')
Versione sug: v3.2.1
```

Copyright © 2026 Paolo Ciccolella. All rights reserved.

```
Modello AI:   7 classi  |  224x224x3  |  2847 KB
Database:     11 diagnosi totali  |  8 reali

Changelog:
  - feat: aggiungi GitHub Publisher nel Pannello Admin
  - fix: pin ai-edge-litert==1.2.0
  ...
```

## 20.8 Prerequisiti

Per il corretto funzionamento del publisher devono essere soddisfatte le seguenti condizioni:

- Git configurato: git config user.name e user.email impostati
- Remote 'origin' puntante al repository GitHub ufficiale
- Accesso push autorizzato: SSH key o token GitHub configurati
- Connessione internet attiva al momento del push
- fpdf2 installato (gia incluso in requirements.txt) per rigenerare il PDF

**[!] ATTENZIONE**

ATTENZIONE — La pubblicazione su GitHub e irreversibile: il tag creato e il push effettuato non possono essere annullati dall'interno del software. Usare la modalita [3] Anteprima per verificare il contenuto prima di procedere. L'operazione [1] Pubblicazione completa richiede una conferma esplicita prima di eseguire il push.

## 20.9 Privacy dei Dati

Il GitHub Publisher e progettato per garantire la piena privacy delle informazioni raccolte in campo. Esiste una distinzione netta tra dati pubblicati (metadati tecnici del software) e dati che rimangono esclusivamente in locale (dati operativi delle analisi). La protezione e implementata a doppio livello: il publisher non include dati operativi nei file generati, e il file .gitignore impedisce strutturalmente che i dati locali vengano mai committati. Dal 2026 questa separazione comprende anche il nuovo monitor visitatori del sito deltaplant.ai: gli eventi analytics first-party, se autorizzati dall'utente del sito, restano nello storage operativo first-party del backend o nel fallback locale e non vengono pubblicati su GitHub. Il pannello Admin puo leggerli via report remoto protetto senza renderli dati versionati.

| | |
|---|---|
| **Pubblicati su GitHub** | Versione software, classi modello AI, shape input, dipendenze, parametri di configurazione, changelog git, dimensione modello. |
| **Rimangono IN LOCALE** | Diagnosi effettuate, conteggi analisi, top classi rilevate, distribuzione livelli di rischio, timestamp diagnosi, immagini acquisite, dati sensori raccolti, eventi analytics first-party del sito e report visitatori esportati dal monitor admin. |
| **Visibili solo in [3] Anteprima** | Le statistiche del database sono consultabili a schermo nel terminale tramite l'opzione [3] Anteprima, ma non vengono mai scritte in nessun file ne trasmesse a servizi esterni. |

## 20.10 Protezione Strutturale tramite .gitignore

Oltre alle politiche del publisher, il file .gitignore nella root del repository esclude permanentemente dal controllo di versione tutti i file che contengono dati operativi locali. Questa protezione e attiva a livello git e non dipende dal publisher: anche in caso di errore o uso diretto di git add, i file elencati non potranno mai essere committati.

| | |
|---|---|
| **delta.db** | Database SQLite principale — contiene tutte le diagnosi, i campioni di |

Copyright © 2026 Paolo Ciccolella. All rights reserved.

| | |
|---|---|
| | fine-tuning e i log operativi. |
| **data/academy_progress.json** | Progressi, punteggi e badge della DELTA Academy dell'operatore corrente. |
| **exports/** | Cartella con i file Excel esportati (delta_diagnoses.xlsx) contenenti lo storico delle diagnosi in formato tabulare. |
| **data/auth.json** | Credenziali amministratore (hash PBKDF2-SHA256 + salt). Sempre escluso dal repo per ragioni di sicurezza. |
| **logs/** | File di log di sistema con timestamp e messaggi di runtime. |
| **data/analytics/** | Database SQLite del monitor visitatori del sito (website_events.sqlite) ed eventuali export JSON locali. In produzione il monitor puo leggere anche il report remoto protetto, ma le copie locali di analytics first-party restano escluse dal repository. |
| **datasets/captures/** | Immagini acquisite dalla camera durante le sessioni di analisi. |

## GARANZIA PRIVACY

Nessun dato agronomico, nessun risultato di analisi e nessuna informazione operativa raccolta dal sistema DELTA viene mai trasmessa a GitHub o a qualsiasi servizio esterno. La protezione opera a due livelli indipendenti: (1) il publisher genera solo metadati tecnici; (2) il .gitignore blocca strutturalmente il commit dei dati locali.

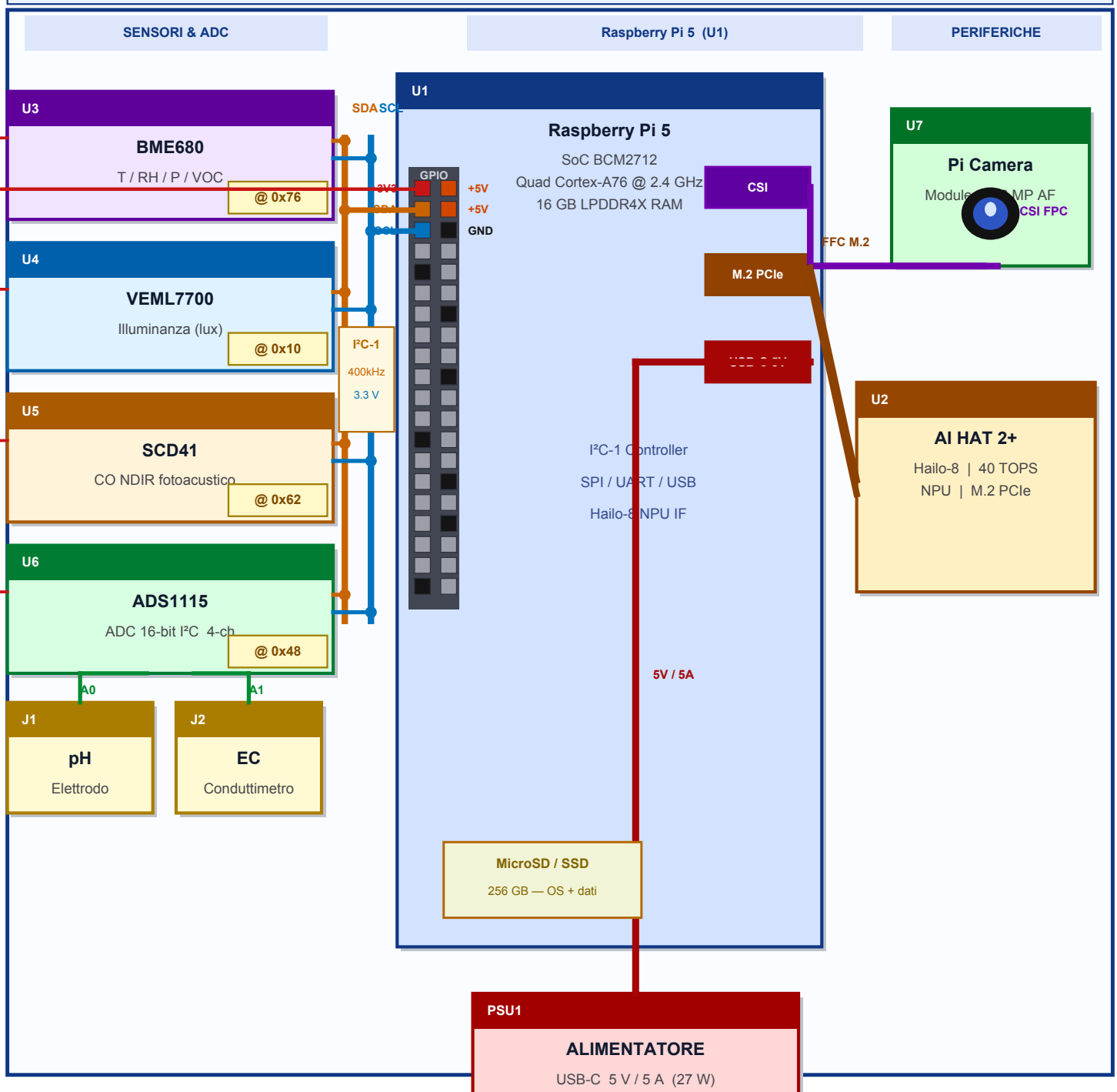
## AUTOMAZIONE COMPLETA

Il GitHub Publisher e progettato per eliminare completamente il lavoro manuale di aggiornamento della documentazione. Ogni push include README, RELEASE e Manuale PDF aggiornati in modo coerente con il software effettivamente in esecuzione su Raspberry Pi.

Copyright © 2026 Paolo Ciccolella. All rights reserved.

# DELTA v3.2 — SCHEMA DI CABLAGGIO COMPLETO

DOC: DELTA-EL-001 | REV. 2.0 | Raspberry Pi 5 | BCM2712 | I²C Bus-1 | DATA: 09/06/2026




## LEGENDA SEGNALI / SIGNAL LEGEND

| +3.3V | +5V | GND | I²C SDA | I²C SCL | CSI FPC | FFC M.2 | Analogico | USB-C PWR |
|---|---|---|---|---|---|---|---|---|
| Pin 1 / 17 | Pin 2 / 4 | Pin 6/9/14… | GPIO2 Pin 3 | GPIO3 Pin 5 | Camera J3 | AI HAT PCIe | A0 pH / A1 EC | 5V 5A 27W |

**NOTE TECNICHE**

Bus I²C-1: 400 kHz Fast-Mode, pull-up 1.8 kΩ a +3.3V. Tutti i sensori alimentati a VCC = 3.3V. CSI: cavo FPC 22-pin (Pi Camera Module 3). FFC AI HAT: 22-pin 0.5 mm pitch (non scollegare a sistema acceso). Alimentatore minimo 27W USB-C (requisito Raspberry Pi 5 + NPU).

Copyright © 2026 Paolo Ciccolella. All rights reserved.

# RENDERING ELETTRICO — CONNESSIONI PIN-BY-PIN E SPECIFICHE

## Tab. 1 — Connessioni GPIO Raspberry Pi 5 (bus I²C-1)

| Dispositivo | Ref. Des. | GPIO / Pin | Segnale | Descrizione | Cavo |
|---|---|---|---|---|---|
| BME680 | U3 | Pin 1 | +3.3V | Alimentazione 3.3 V | Dupont rosso |
| BME680 | U3 | Pin 6 | GND | Massa comune | Dupont nero |
| BME680 | U3 | Pin 3 | SDA | I²C SDA — GPIO2 (addr 0x76) | Dupont ambra |
| BME680 | U3 | Pin 5 | SCL | I²C SCL — GPIO3 (addr 0x76) | Dupont blu |
| VEML7700 | U4 | Pin 1 | +3.3V | Alimentazione 3.3 V | Dupont rosso |
| VEML7700 | U4 | Pin 6 | GND | Massa comune | Dupont nero |
| VEML7700 | U4 | Pin 3 | SDA | I²C SDA — GPIO2 (addr 0x10) | Dupont ambra |
| VEML7700 | U4 | Pin 5 | SCL | I²C SCL — GPIO3 (addr 0x10) | Dupont blu |
| SCD41 | U5 | Pin 1 | +3.3V | Alimentazione 3.3 V | Dupont rosso |
| SCD41 | U5 | Pin 6 | GND | Massa comune | Dupont nero |
| SCD41 | U5 | Pin 3 | SDA | I²C SDA — GPIO2 (addr 0x62) | Dupont ambra |
| SCD41 | U5 | Pin 5 | SCL | I²C SCL — GPIO3 (addr 0x62) | Dupont blu |
| ADS1115 | U6 | Pin 1 | +3.3V | Alimentazione 3.3 V | Dupont rosso |
| ADS1115 | U6 | Pin 6 | GND | Massa comune | Dupont nero |
| ADS1115 | U6 | Pin 3 | SDA | I²C SDA — GPIO2 (addr 0x48) | Dupont ambra |
| ADS1115 | U6 | Pin 5 | SCL | I²C SCL — GPIO3 (addr 0x48) | Dupont blu |
| ADS1115 ch.A0 | J1 | ADS1115 A0 | AIN0 | Elettrodo pH — segnale 0–3 V (potenziome | Dupont verde |
| ADS1115 ch.A1 | J2 | ADS1115 A1 | AIN1 | Cella EC — conducibilità 0–5 mS/cm → 0–3 | Dupont verde |
| Pi Camera Mod.3 | U7 | CSI J3 | CSI | Cavo FPC 22-pin Camera Serial Interface | FPC flat |
| AI HAT 2+ | U2 | M.2 B+M | PCIe | Slot M.2 PCIe + cavo FFC 22-pin 0.5 mm | FFC 0.5 mm |
| Alimentatore | PSU1 | — | 5V/5A | USB-C 5V / 5A (minimo 27W) per Raspber | USB-C |

## Tab. 2 — Mappa Indirizzi I²C (Bus-1 | GPIO2 / GPIO3)

| | |
|---|---|
| **BME680 (U3) — T/RH/P/VOC** | 0x76 (default, SDO=GND). Alt. 0x77 se SDO=VCC. Freq. campionamento: configurabile via OS. |
| **VEML7700 (U4) — Lux** | 0x10 — fisso, non modificabile. Integrazione: 100 ms (default). Range: 0.0036 – 120.000 lux. |
| **SCD41 (U5) — CO** | 0x62 — fisso, non modificabile. Attesa post-power-up: ≥1 000 ms prima del primo comando. |
| **ADS1115 (U6) — ADC 16-bit** | 0x48 (default, ADDR=GND). Alt. 0x49/0x4A/0x4B per ADDR=VCC/SDA/SCL. Canali A0=pH, A1=EC, A2/A3 liberi. |
| **Freq. bus I²C-1** | 400 kHz (Fast Mode). Supportata da tutti i sensori del bus. |
| **Pull-up resistori** | 1.8 kΩ su SDA e SCL a +3.3V. Interni a Raspberry Pi 5 o da aggiungere esternamente se bus lungo. |
| **Lunghezza max. bus** | < 30 cm con Dupont standard. Per distanze maggiori: buffer I²C P82B96. |
| **Verifica bus (shell)** | sudo i2cdetect -y 1 → mostra indirizzi di tutti i dispositivi presenti. |

Copyright © 2026 Paolo Ciccolella. All rights reserved.

## Tab. 3 — Caratteristiche Elettriche e Budget di Potenza

| | |
|---|---|
| **Raspberry Pi 5 (U1)** | VCC = 5V via USB-C. Consumo: 3–5 W idle, 12–15 W carico AI. Corrente max assorbita: 5A. |
| **AI HAT 2+ Hailo-8 (U2)** | Alimentato via M.2 PCIe (FFC). Consumo NPU: 2.5 W @ 40 TOPS. TDP max: 5W. |
| **BME680 (U3)** | VCC = 1.7–3.6 V. $I_a$ = 3.1 mA (misura), 0.15 µA (sleep). Consumo tipico continuo: < 1 mW. |
| **VEML7700 (U4)** | VCC = 2.5–3.6 V. $I_a$ = 90 µA (attivo), 3 µA (standby). |
| **SCD41 (U5)** | VCC = 2.4–5.5 V. $I_a$ = 205 mA (misura), 0.5 mA (idle). Riscaldamento interno: no (NDIR fotoacustico). |
| **ADS1115 (U6)** | VCC = 2.0–5.5 V. $I_a$ = 150 µA (continuo), 2 µA (power-down). |
| **Elettrodo pH (J1)** | Passivo — nessuna alimentazione diretta. Vout ±0.5 V (pH 4–10, Vref = GND). Impedenza: > 100 MΩ. |
| **Cella EC (J2)** | Alimentazione AC interna all'ADS1115. Segnale: 0–3.3 V proporzionale a 0–5 mS/cm. |
| **Budget totale picco** | ≈ 20–22 W. Alimentatore minimo raccomandato: 27W USB-C originale Raspberry Pi. |

**NOTE DI PROGETTAZIONE ELETTRICA (Design Engineering Notes)**

- Tutti i sensori I²C devono condividere la stessa massa (GND) con Raspberry Pi 5 (loop di terra causano offset di misura).
- Cavi Dupont raccomandati < 20 cm (capacità parassita max bus I²C: 400 pF totali).
- SCD41: attendere ≥ 1 000 ms dal power-up prima del primo comando di misura.
- Elettrodo pH: richiedere calibrazione bipoint (pH 4.0 e pH 7.0) prima dell'uso.
- AI HAT 2+: non scollegare il cavo FFC M.2 a sistema in funzione (rischio danni permanenti alla NPU e/o al connettore).
- Proteggere l'elettronica da umidità > 85% RH: condensazione può causare corto circuito.
- EMC: separare fisicamente i cavi di alimentazione (5V/USB-C) dai cavi di segnale I²C e analogici lungo tutto il percorso fino al connettore GPIO.

**[!] ATTENZIONE**

SICUREZZA HARDWARE — Non invertire VCC e GND sui connettori Dupont: i sensori BME680, VEML7700 e SCD41 non sono protetti da inversione di polarità. Controllare sempre la pin-out del datasheet prima di collegare. Verificare con: sudo i2cdetect -y 1 (nessun dispositivo = cavo invertito o aperto).

Copyright © 2026 Paolo Ciccolella. All rights reserved.

# Appendice Licenza — DELTA PLANT SOFTWARE LICENSE

Copyright © 2026 Paolo Ciccolella. All rights reserved.

Software Release: v3.2

(This release field must be updated at each official software release.)

Il testo integrale della licenza e disponibile nel file LICENSE nella directory radice del repository GitHub. GitHub visualizza automaticamente la licenza nel pannello informativo del progetto.

```
> POSIZIONE FILE LICENSE
DELTA-PLANT/
  LICENSE   <- testo integrale della licenza
```

## 1. Core System (Proprietary)

The DELTA PLANT core system, including but not limited to:

- main control software
- sensor management system
- automation engine
- safety and execution modules

is proprietary software.

You are NOT permitted to:

- copy, modify, distribute, or reverse engineer the core system
- use the core system outside the authorized DELTA ecosystem
- remove or alter copyright notices

Any unauthorized use of the core system is strictly prohibited.

## 2. Modules / Extensions (Open Source Components)

Certain modules, plugins, or SDK components of DELTA PLANT may be released under open-source licenses (such as MIT or Apache 2.0).

These components are clearly marked in their respective directories.

For open-source modules:

- you may use, modify, and distribute them
- you must respect the license specified in each module
- attribution to the original author is required

## 3. AI Services and Cloud Features

Any AI-related services, cloud APIs, or external model integrations (including but not limited to language model processing) are provided as a service layer and are NOT part of the open-source components.

These services may:

- require authentication

- be subject to usage limits
- be modified or discontinued at any time

## 4. Liability Disclaimer

**[!] ATTENZIONE**

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND.

The author is not responsible for:

- hardware damage
- data loss
- incorrect automation behavior
- misuse of the system

## 5. Scientific Research Use

Scientific and academic research use of the DELTA PLANT core system is allowed only for non-commercial purposes, subject to all terms in this license.

For research use, you must:

- keep all copyright and license notices intact
- provide clear attribution to the author in publications and reports
- use the software in compliance with applicable laws and ethical standards
- avoid redistributing proprietary core code or derivative proprietary builds

Research use does NOT grant ownership or relicensing rights on the DELTA PLANT proprietary core system.

## 6. Commercial Use

Commercial use of the DELTA PLANT core system requires a separate written license agreement with the author.

Open-source modules may be used commercially according to their respective licenses.

## 7. Final Terms

By using any part of DELTA PLANT, you agree to these terms.

Violation of this license may result in termination of usage rights and legal action.

---

*END OF LICENSE*

Copyright © 2026 Paolo Ciccolella. All rights reserved.